

# FAST2: Adaptive Geometric Indexing and Operation Reordering for Lazy Multi-Valued Heuristic Search under Dimensionality Reduction

FAST — Fast Adaptive Search with Trees for multi-valued heuristic search (L-NAMOA\*dr-FAST2)

*Research paper draft, September 2026. Every measurement is taken from archived run artifacts (Appendix A); no new search experiments were run for this document. Archived solution files were re-verified independently (Section 5.7).*

---

## Abstract

---

Multi-valued heuristics (MVHs) supply multi-objective search with set-valued lower bounds, and dimensionality reduction (DR) removes one dimension from every dominance test. L-NAMOA\*dr-mvh (Wolff, Felner, and Salzman 2026) combines the two lazily and requires only an admissible MVH. Its running time is dominated by two operations: dominance tests against dense per-state and goal frontiers, and the sequential scan of the heuristic set  $H(s)$  that binds each path to one heuristic vector. We present FAST2, a framework that accelerates both without changing any search decision. FAST2 indexes truncated frontiers adaptively, promoting a contiguous array to a dynamic k-d tree when a measured scan-cost model predicts a gain; it evaluates the local dominance test before heuristic selection, which removes 67–80% of selection calls; it captures full-dimensional witnesses during truncated scans and caches goal-frontier witnesses per heuristic vector; and it maintains the DR monotonicity certificate exactly. We prove that FAST2 reproduces the baseline trajectory. We also make explicit a precondition that the literature leaves implicit: under DR, each  $H(s)$  must be scanned in non-decreasing order of its first component (with lexicographic tie-breaking) for the search to be complete and sound with genuine MVHs. The precondition is unnecessary for landmark families, for which component-wise maximum aggregation is admissible and expands 11–41% fewer nodes. On 81 archived inputs with  $M \in \{3, 4, 5, 6\}$  objectives that were solved by two or more solvers, all solvers return identical solution sets. On DIMACS New York subgraphs with genuine A\*pex MVHs, FAST is 2.6–89× faster than the baseline (geometric means 6.0×, 31×, and 77× at  $M = 4, 5, 6$ ); the goal-witness cache of FAST2 removes a further 12–38% in a same-binary comparison; and five instances with up to 334,791 Pareto-optimal solutions are solved in 74–497 s where the baseline exceeds 900 s. Finally, we explain why a static k-d index over  $H(s)$ , which is 1.3–1.9× faster than a linear scan in the baseline’s operation order, stops paying once that order is changed.

---

## 1 Introduction and Problem Setting

---

### 1.1 Multi-objective search and the frontier explosion

In the multi-objective shortest-path problem (MO-SPP) every edge carries a vector of  $M$  non-negative costs, and the task is to return one path for every non-dominated cost vector between a start and a goal state (Salzman et al. 2023; Salzman et al. 2026). Best-first algorithms of the multi-objective A\* family (Stewart and White 1991; Mandow and Pérez de la Cruz 2005, 2010) keep, at every state, the set of non-dominated cost-to-come vectors found so far, and test every generated and extracted path against it (a *local* test) and against the set of solutions found so far (a *global* test). The number of Pareto-optimal solutions can grow exponentially with the graph; on the road subgraphs studied here it reaches 334,791 at  $M = 6$ . Dominance tests therefore dominate running time as  $M$  grows (Pulido, Mandow, and Pérez-de-la-Cruz 2015; Ren et al. 2025; Shperberg et al. 2026). Bi-objective search removes most of this cost by exploiting a total order (BOA\*, Hernández et al. 2023a), and lazy dominance checking reduces it for more objectives (Hernández et al. 2023b); for  $M \geq 4$ , however, no index that locates

candidates through a single-coordinate order can avoid testing every stored vector in the worst case, whereas a geometric index answers the equivalent orthant-emptiness query in  $O(n^{1-1/d})$  for  $d = M - 1$  (Shperberg et al. 2026).

## 1.2 Multi-valued heuristics and dimensionality reduction

A single-valued heuristic (SVH) maps a state to one lower-bound vector. A multi-valued heuristic maps it to a set of mutually non-dominated vectors, which can represent the trade-offs of the remaining path (Zhang et al. 2023; Geißer et al. 2022; Skyler et al. 2024). Dimensionality reduction (Pulido, Mandow, and Pérez-de-la-Cruz 2015) relies on lexicographic extraction from OPEN: under a consistent SVH, the first cost component of the paths extracted at a state never decreases, so dominance tests may ignore it. Wolff, Felner, and Salzman (2026) showed that this invariant fails under MVHs, because two paths to the same state may be evaluated with different heuristic vectors. Their algorithm L-NAMOA\*dr-mvh restores correctness lazily: it binds each path to one heuristic vector at a time (CHOOSEH), re-binds it when the goal frontier refutes the current vector, applies DR optimistically, and falls back to a full-dimensional test when the monotonicity assumption is violated.

## 1.3 The dual bottleneck

Two operations dominate L-NAMOA\*dr-mvh.

**(B1) Frontier dominance.** Every extraction and every generation queries the truncated frontier of a state, and every extraction and every heuristic selection queries the truncated goal frontier  $T$ . The reference implementation scans these sets linearly. On our instances  $|T|$  reaches  $10^4$ – $10^5$  and per-state frontiers reach thousands of vectors.

**(B2) Sequential heuristic evaluation.** CHOOSEH scans  $H(s)$  in lexicographic order and tests every candidate  $g + h$  against  $T$  until one survives. On A\*pex MVHs  $|H(s)|$  reaches 32,966. Profiled on contiguous storage, CHOOSEH takes 33–66% of the running time on grids with landmark MVHs and 77% on a grid with an A\*pex MVH (Section 5).

## 1.4 Contributions

1. **Theory (Section 2).** We distinguish *universally valid* heuristic families (landmarks), for which component-wise maximum aggregation is admissible and prunes strictly more than set semantics, from *genuine* MVHs, for which maximum aggregation is inadmissible. We prove that scanning each  $H(s)$  in non-decreasing order of  $h_1$  makes the truncated goal tests of L-NAMOA\*dr-mvh complete, and that lexicographic tie-breaking makes them sound; that the order is irrelevant for landmark families; and that it cannot be dropped for genuine MVHs (a three-state counterexample, confirmed on archived runs).
  2. **Framework (Section 3).** FAST2 combines adaptive frontier indexing, an optional static heuristic index, local-first operation reordering, full-dimensional witness capture, goal-witness caching, redundancy pointers, and exact monotonicity tracking. We prove that FAST2 executes the same sequence of OPEN operations as the baseline.
  3. **Analysis (Section 4).** Every successful heuristic selection must pay one negative frontier query; an aggregate box test over a group of heuristic vectors is exact only when the group has a least (discard rule) or greatest (accept rule) member after truncation, and the accept rule, the only one that shortens certification, almost never fires; and a heuristic tree is a tree of oracle calls, whereas a frontier tree is a tree of comparisons. Reordering removes most of the long scans that a heuristic tree was designed to shorten.
  4. **Empirical synthesis (Section 5).** We consolidate all archived runs (grids and DIMACS road networks,  $M = 3$  to 6, A\*pex approximation factors  $\varepsilon \in \{0, 0.01, 0.05, 0.1\}$ ) and re-verify 274 archived solution files.
-

## 2 Theoretical Foundations: MVH versus SVH

### 2.1 Notation

A MOS instance is  $P = \langle S, E, c, s_{\text{start}}, s_{\text{goal}} \rangle$  with  $c : E \rightarrow \mathbb{R}_{\geq 0}^M$ . For  $x, y \in \mathbb{R}^M$ ,  $x$  *weakly dominates*  $y$ , written  $x \preceq y$ , if  $x_k \leq y_k$  for all  $k$ ;  $x \prec y$  if  $x \preceq y$  and  $x \neq y$ .  $x \leq_{\text{lex}} y$  if  $x = y$  or  $x_k < y_k$  at the first index  $k$  where they differ.  $\text{Nd}(X)$  is the cost-unique non-dominated subset of  $X$ , and  $\Pi^*$  is the cost-unique Pareto-optimal solution set. A search node  $n$  has a state  $s(n)$ , a path cost  $g(n)$ , a heuristic vector  $h(n) \in H(s(n))$  with index  $i(n)$ , and  $f(n) = g(n) + h(n)$ .  $\Sigma(s)$  is the set of paths from  $s$  to  $s_{\text{goal}}$ .

The **truncation**  $\text{Tr}(x) = (x_2, \dots, x_M)$  removes the first component;  $d = M - 1$ . For a set  $X$  of full vectors,

$$X \preceq_{\text{Tr}} x \iff \exists y \in X : \text{Tr}(y) \preceq \text{Tr}(x),$$

read “ $x$  is Tr-dominated by  $X$ ”. For every state the search keeps two closed sets (Wolff et al. 2026):  $G_{\text{cl}}(s)$ , all path costs expanded at  $s$  (append-only), and the truncated frontier  $F(s)$  ( $G_{\text{cl}}^{\text{Tr}}(s)$  in their notation), which is updated on the expansion of  $g$  by removing every  $p$  with  $\text{Tr}(g) \preceq \text{Tr}(p)$  and inserting  $g$ . The members of  $F(s)$  keep their full vectors.  $T \equiv F(s_{\text{goal}})$  is the truncated goal frontier, and  $m(s) = \max\{p_1 : p \in F(s)\}$ .

### 2.2 Universal and existential validity

**Definition 1.** A vector  $h$  is *universally valid* at  $s$  if  $h \preceq c(\sigma)$  for every  $\sigma \in \Sigma(s)$ . An MVH  $H$  is *admissible* if for every  $s$  and every  $\sigma \in \Sigma(s)$  some  $h \in H(s)$  satisfies  $h \preceq c(\sigma)$  (Wolff et al. 2026; Skyler et al. 2026a).  $H(s)$  is a *landmark family* if all its members are universally valid, and *genuine* otherwise.

Landmark (ALT) bounds (Goldberg and Harrelson 2005), applied per objective on a symmetric graph, give  $h_{\ell}(s) = (|d_k(s, \ell) - d_k(s_{\text{goal}}, \ell)|)_{k=1..M}$ ; each such vector is universally valid by the triangle inequality, and a set of  $K$  of them is a landmark family. An A\*pex MVH (Zhang et al. 2022; Wolff et al. 2026) stores, per state, the component-wise minima (“apexes”) of groups of paths found by a backward approximate search; an apex bounds the paths of its group only, so the set is in general genuine. For  $\varepsilon = 0$  the backward search is exact and  $H(s)$  is the Pareto set of costs-to-go.

L-NAMOA\*dr-mvh evaluates a path  $(s, g)$  *existentially*: the path is discarded only when every  $h \in H(s)$  is refuted, i.e.,  $T \preceq_{\text{Tr}} g + h$  for all  $h \in H(s)$ .

**Proposition 1 (Maximum aggregation of landmark families).** Let  $H(s)$  be a landmark family and  $\hat{h}(s) = \max H(s)$ , taken component-wise. Then (i)  $\hat{h}(s)$  is universally valid, and (ii) for every  $T$  and  $g$ , if  $T \preceq_{\text{Tr}} g + h$  for some  $h \in H(s)$ , then  $T \preceq_{\text{Tr}} g + \hat{h}(s)$ .

*Proof.* (i) Every member is dominated by  $c(\sigma)$  for every  $\sigma \in \Sigma(s)$ , hence so is their component-wise maximum. (ii)  $h \preceq \hat{h}(s)$  and  $\preceq$  is transitive.  $\square$

At equal  $T$ , the single vector  $\hat{h}(s)$  discards a path as soon as *one* member is refuted, while the existential rule waits until *all* are. Each additional landmark vector is one more alternative that can keep a path alive, so enlarging a landmark family weakens existential pruning. Table 1 confirms both effects on archived runs: on  $8 \times 8$  grids expansions grow from  $K = 1$  to  $K = 64$  landmark vectors, and replacing each family by its maximum reduces expansions by 11–41% while returning the same front.

**Table 1.** Landmark families: set size and maximum aggregation. Expansions are identical for the baseline and FAST on every row; fronts are identical within each instance.

| instance                          | M | $ T $ | exp., $K = 1$ | exp., $K = 64$ | exp., $K$ vectors (set) | exp., maximum | reduction |
|-----------------------------------|---|-------|---------------|----------------|-------------------------|---------------|-----------|
| grid $8 \times 8$ , $\rho = -0.6$ | 3 | 428   | 6,757         | 7,193          | 7,193 ( $K=64$ )        | 5,209         | 27.6%     |

| instance                | M | T      | exp., $K = 1$ | exp., $K = 64$ | exp., $K$ vectors (set) | exp., maximum | reduction |
|-------------------------|---|--------|---------------|----------------|-------------------------|---------------|-----------|
| grid 8×8, $\rho=-0.6$   | 4 | 6,404  | 47,433        | 49,242         | 49,242 ( $K=64$ )       | 42,623        | 13.4%     |
| grid 15×15, $\rho=-0.6$ | 3 | 2,776  | –             | –              | 170,052 ( $K=200$ )     | 141,634       | 16.7%     |
| grid 10×10, $\rho=-0.6$ | 4 | 12,667 | –             | –              | 187,070 ( $K=50$ )      | 166,134       | 11.2%     |
| grid 12×12, $\rho=0.0$  | 4 | 13,709 | –             | –              | 214,670 ( $K=50$ )      | 174,237       | 18.8%     |
| grid 10×10, $\rho=-0.3$ | 5 | 48,950 | –             | –              | 448,472 ( $K=50$ )      | 394,386       | 12.1%     |
| grid 10×10, $\rho=0.3$  | 6 | 17,164 | –             | –              | 197,696 ( $K=25$ )      | 116,863       | 40.9%     |

The baseline benefits most: with the maximum vector it solved the  $M = 4$  grids in 2.1–3.5 s instead of 7.0–9.6 s, and the  $M = 5$  and  $M = 6$  grids in 29 s and 18 s, where the set version exceeded its 60 s limit.

**Proposition 2 (Maximum aggregation is inadmissible for genuine MVHs).** There are admissible genuine MVHs  $H$  for which  $\max H(s)$  is not admissible.

*Proof.* Let the paths from  $s$  cost  $(1, 10)$  and  $(11, 2)$ , and let  $H(s) = \{(1, 8), (9, 2)\}$ , an admissible MVH (Skyler et al. 2026a). Then  $\max H(s) = (9, 8) \not\preceq (1, 10)$ , and a Pareto-optimal solution whose suffix costs  $(1, 10)$  can be pruned.  $\square$

The single-objective analogue is  $kA^*$  for one-to-many search (Stern et al. 2021): with admissible per-goal heuristics, an aggregation function yields an admissible search if and only if it never exceeds the minimum (their Theorem 3), and maximum aggregation is inadmissible. A genuine MVH relates to the Pareto-optimal completions of a path as per-goal heuristics relate to goals: a member bounds only the completions it covers. Given only  $H(s)$ , the largest single vector that is admissible for every compatible set of completions is the component-wise minimum (the ideal point of  $H(s)$ ), because the completions may coincide with  $H(s)$  itself. This is the key that C-MVH-Min\* uses to order nodes under Min ordering (Anonymous 2026b).

Landmark families are therefore better served by a single aggregated vector; genuine MVHs must be evaluated existentially. The rest of this paper concerns genuine MVHs.

### 2.3 Lazy dimensionality reduction

L-NAMOA\*dr-mvh (Wolff et al. 2026, Algorithm 2) orders OPEN by  $\leq_{\text{lex}}$  on  $f$ . On extraction of  $n$  it first applies the goal test  $T \preceq_{\text{Tr}} f(n)$ ; if the test succeeds, it re-binds  $n$  to

$$\text{Choose}H(s, g, j) = \min\{i \geq j : \neg(T \preceq_{\text{Tr}} g + h^i)\} \quad (\perp \text{ if none}),$$

with  $j = i(n) + 1$ , and re-inserts it. Otherwise it applies the local test

$$\text{LocalDomCheck}(s, g) = \begin{cases} \text{false} & \text{if } \neg(F(s) \preceq_{\text{Tr}} g), \\ \text{true} & \text{else if } m(s) \leq g_1, \\ \exists g' \in G_{\text{cl}}(s) : g' \preceq g & \text{otherwise,} \end{cases}$$

expands  $n$ , and generates each successor  $(s', g')$  with  $\text{Choose}H(s', g', 1)$  followed by  $\text{LocalDomCheck}(s', g')$ .

**Lemma 1 (Exactness of the lazy local test).**  $\text{LocalDomCheck}(s, g) = [\exists g' \in G_{\text{cl}}(s) : g' \preceq g]$ .

*Proof.* Every  $g' \in G_{\text{cl}}(s)$  entered  $F(s)$  when it was expanded. A member leaves  $F(s)$  only when an inserted vector Tr-dominates it, so by induction some current member  $p$  satisfies  $\text{Tr}(p) \preceq \text{Tr}(g')$ . Hence, if  $g' \preceq g$  for some  $g' \in G_{\text{cl}}(s)$ , then  $F(s) \preceq_{\text{Tr}} g$ , and the first case cannot return false. In the second case every Tr-dominator  $p \in F(s)$  satisfies  $p_1 \leq m(s) \leq g_1$ , so  $p \preceq g$  and  $p \in G_{\text{cl}}(s)$ . The third case is exact.  $\square$

The proof uses only  $m(s) \geq \max\{p_1 : p \in F(s)\}$ . An underestimate is unsound under MVHs: the most recently inserted vector need not carry the largest first component, because extraction order no longer implies monotone  $g_1$  at a state (Wolff et al. 2026, Lemma 1). An overestimate is sound but triggers unnecessary fallbacks.

**Lemma 2 (Monotone truncated region of  $T$ ).** The set  $R(T) = \{x : T \preceq_{\text{Tr}} x\}$  never shrinks, and any vector that has ever been a member of  $T$  and Tr-dominates  $x$  certifies  $x \in R(T)$ .

*Proof.* A member  $t$  leaves  $T$  only when an inserted  $t'$  satisfies  $\text{Tr}(t') \preceq \text{Tr}(t)$ ; apply transitivity.  $\square$

## 2.4 The lexicographic scan as a correctness precondition

Wolff et al. (2026) state CHOOSEH as a scan of  $H(s)$  “in lexicographic order”, and the KD-ChooseH manuscript writes the selected vector as  $\min_{\text{lex}}$  of the surviving vectors (Anonymous 2026a). The order is not a tie-breaking convention. It is the only place where the correctness of the truncated goal tests depends on the heuristic.

**Theorem 1 (Scan order).** Let  $H$  be admissible, and let every  $H(s) = \langle h^1, \dots, h^{k(s)} \rangle$  be scanned in its stored order.

1. If every  $H(s)$  is stored in non-decreasing order of  $h_1$ , then every vector  $t$  that enters  $T$  before an undiscovered  $c^* \in \Pi^*$  satisfies  $t_1 \leq c_1^*$ . Consequently the truncated goal tests (goal test and CHOOSEH) never discard a path that extends to an undiscovered Pareto-optimal solution, and L-NAMOA\*dr-mvh is complete.
2. If, in addition, ties in  $h_1$  are broken lexicographically (every  $H(s)$  is sorted by  $\leq_{\text{lex}}$ ), no dominated solution enters Sols.

*Proof sketch.* Fix an undiscovered  $c^*$ . Call  $n$  a *witness* for  $c^*$  if some suffix  $\sigma \in \Sigma(s(n))$  satisfies  $g(n) + c(\sigma) = c^*$ ; admissibility gives  $h^* \in H(s(n))$  with  $g(n) + h^* \preceq c^*$ . We show by induction over iterations that OPEN always contains a witness  $n$  with  $f_1(n) \leq c_1^*$ , and with  $f(n) \leq_{\text{lex}} c^*$  under the assumption of part 2.

- a. *Refutation safety.* Suppose  $t \in T$  with  $\text{Tr}(t) \preceq \text{Tr}(g(n) + h^*)$ . When  $t$  was extracted, OPEN contained a witness  $n'$  with  $f_1(n') \leq c_1^*$  (induction hypothesis); admissibility forces  $H(s_{\text{goal}}) = \{0\}$ , so the extracted goal node had  $f = t$ . Extraction is lexicographic, so  $t \leq_{\text{lex}} f(n')$  and  $t_1 \leq f_1(n') \leq c_1^*$ . Together with  $\text{Tr}(t) \preceq \text{Tr}(c^*)$  this gives  $t \preceq c^*$ , and Pareto optimality forces  $t = c^*$ , contradicting that  $c^*$  is undiscovered. Hence  $h^*$  is never refuted while  $c^*$  is undiscovered.
- b. *Selection.* An index skipped by CHOOSEH was refuted when it was skipped and stays refuted (Lemma 2), so the index of  $h^*$  is never skipped, and CHOOSEH returns an index  $i$  no later than that of  $h^*$ . Under part 1 this gives  $h_1^i \leq h_1^*$  and  $f_1 = g_1 + h_1^i \leq c_1^*$ ; under part 2 it gives  $h^i \leq_{\text{lex}} h^*$  and  $f = g + h^i \leq_{\text{lex}} g + h^* \leq_{\text{lex}} c^*$ .
- c. *Maintenance.* Re-evaluation and expansion of a witness produce, by (b), a re-inserted node or a successor witness with the same bound. If the local test discards the witness, Lemma 1 yields  $g' \in G_{\text{cl}}(s)$  with  $g' \preceq g(n)$ , and Pareto optimality forces  $g' + c(\sigma) = c^*$ ; the argument continues along the successors of the node that expanded  $g'$ , as in the completeness proof of Wolff et al. (2026).

Soundness (part 2): if a solution  $x$  is dominated by  $c^*$ , then  $c^* <_{\text{lex}} x$ , so the witnesses of  $c^*$  precede  $x$  in OPEN,  $c^*$  enters  $T$  first, and  $x$  is refuted by the goal test.  $\square$

The lexicographic refinement of part 2 settles ties in  $f_1$ , which would otherwise allow a dominated solution with  $x_1 = c_1^*$  to be extracted before  $c^*$ . Because the proof must bound  $h_1$  against an *unknown* surviving vector  $h^*$ , only the survivor with minimum  $h_1$  is safe: under DR the selection rule is fixed up to ties. The A\*pex builder of Wolff et al. writes every  $H(s)$  in the extraction order of its backward search, and the seven archived A\*pex heuristic files we inspected contain no  $h_1$  inversion; the reference implementation depends on this property without checking it. Ties in  $h_1$  are not always broken lexicographically (9–273 of 3,000 states in each New York file), yet every archived output on these files is an antichain (Section 5.7).

**Proposition 3 (Landmark families are order-free).** If every  $H(s)$  is a landmark family, the conclusions of Theorem 1 hold for every scan order.

*Proof.* Every member satisfies  $h \preceq c(\sigma)$  for every witness suffix, so  $f = g + h \preceq c^*$  and  $f \leq_{\text{lex}} c^*$  for any selected member; step (b) no longer needs the order.  $\square$

**Proposition 4 (Necessity for genuine MVHs).** For genuine MVHs, a scan order that is not non-decreasing in  $h_1$  can make L-NAMOA\*dr-mvh incomplete or unsound.

*Proof.* Consider the instance of Table 2, with  $H(s_1)$  scanned as  $\langle (12, 0), (1, 6) \rangle$ .  $H$  is admissible: the only path from  $s_1$  costs  $(1, 6)$ , which  $(1, 6)$  covers.  $(12, 0)$  is a member of an admissible set, but it is not a lower bound for that path. In variant A the search extracts  $(10, 5)$  at  $s_{\text{goal}}$  before  $s_1$ , because  $f(s_1) = (13, 1)$ ; the successor of  $s_1$  at the goal, with cost  $(2, 7)$ , is then refuted by  $\text{Tr}(10, 5) = 5 \leq 7$  and lost. In variant B the vector  $(10, 9)$  is extracted first and kept;  $(2, 7)$  is not refuted, because  $9 > 7$ , and both vectors are returned, although  $(2, 7) \prec (10, 9)$ . With the lexicographic order  $\langle (1, 6), (12, 0) \rangle$ ,  $f(s_1) = (2, 7)$  is extracted first and both variants are solved correctly.  $\square$

**Table 2.** A three-state counterexample. Edges:  $s_{\text{start}} \rightarrow s_{\text{goal}}$  with cost  $x$ ;  $s_{\text{start}} \rightarrow s_1$  with cost  $(1, 1)$ ;  $s_1 \rightarrow s_{\text{goal}}$  with cost  $(1, 6)$ .  $H(s_{\text{start}}) = H(s_{\text{goal}}) = \{(0, 0)\}$ ,  $H(s_1) = \{(1, 6), (12, 0)\}$ .

| variant | $x$       | $\Pi^*$               | output, scan $\langle (12, 0), (1, 6) \rangle$ | failure    | output, lexicographic scan |
|---------|-----------|-----------------------|------------------------------------------------|------------|----------------------------|
| A       | $(10, 5)$ | $\{(2, 7), (10, 5)\}$ | $\{(10, 5)\}$                                  | incomplete | $\{(2, 7), (10, 5)\}$      |
| B       | $(10, 9)$ | $\{(2, 7)\}$          | $\{(10, 9), (2, 7)\}$                          | unsound    | $\{(2, 7)\}$               |

The archived order experiments show the same behaviour at scale (Table 3). On seven landmark grids, sum-ascending, sum-descending, file-order, zero, and maximum variants of every  $H(s)$  return identical fronts (Proposition 3). On the  $M = 4$  grid with an A\*pex MVH, whose front has 11,330 vectors, the sum-descending order returns 16,993 vectors and the sum-ascending order 7,534; neither output is an antichain (Section 5.7), and the reference implementation returns the same 16,993 vectors on the reordered input, so the failure is algorithmic. On the  $M = 5$  road subgraph nyr3k ( $\varepsilon = 0.05$ ), selecting the survivor closest to the ideal point misses 15 of the 4,313 optimal vectors. Removing DR restores order independence (Table 3, lower part): a full-dimensional variant returns the exact front under all four orders tested. The  $h_1$ -sorted file order is also the cheapest: it binds the smallest valid  $f_1$  and never delays a witness.

**Table 3.** Scan order under DR (upper part) and without DR (lower part). “File order” is the order written by the heuristic builder (non-decreasing  $h_1$  for A\*pex MVHs, arbitrary for landmark MVHs). “Blind” is search with  $h = 0$ .

| instance                                | MVH      | scan order                               | output             | exp.      | output correct                    |
|-----------------------------------------|----------|------------------------------------------|--------------------|-----------|-----------------------------------|
| grid 10×10 (M=4)                        | A*pex    | file order                               | 11,330             | 47,305    | yes (equals blind search)         |
| grid 10×10 (M=4)                        | A*pex    | sum descending                           | 16,993             | 72,898    | no (contains dominated vectors)   |
| grid 10×10 (M=4)                        | A*pex    | sum ascending                            | 7,534              | 34,499    | no (not an antichain, incomplete) |
| 7 grids (M=3–6)                         | landmark | file / sum asc. / sum desc. / max / zero | identical per grid | vary      | yes                               |
| nyr3k (M=5, $\varepsilon=0.05$ )        | A*pex    | ideal point first                        | 4,298 of 4,313     | –         | no (15 missing)                   |
| nyr3k (M=5, $\varepsilon=0.05$ ), no DR | A*pex    | file order                               | 4,313              | 48,729    | yes                               |
| nyr3k (M=5, $\varepsilon=0.05$ ), no DR | A*pex    | sum ascending                            | 4,313              | 48,845    | yes                               |
| nyr3k (M=5, $\varepsilon=0.05$ ), no DR | A*pex    | sum descending                           | 4,313              | 293,378   | yes                               |
| nyr3k (M=5, $\varepsilon=0.05$ ), no DR | A*pex    | random                                   | 4,313              | 311,481   | yes                               |
| nyr3k (M=5, $\varepsilon=0.05$ ), blind | –        | –                                        | 4,313              | 1,504,044 | reference                         |

With DR and the file order, FAST2 expands 48,728 nodes on the same instance.

### 3 The FAST2 Algorithmic Framework

---

#### 3.1 Design principle and search state

FAST2 changes how each predicate of L-NAMOA\*dr-mvh is evaluated, never what it returns. Every data structure answers the existence predicates of Section 2.3 over the same live sets as the reference implementation, so FAST2 inherits the correctness theorem of Wolff et al. (2026) and Theorem 1. We write **FAST** for the configuration without the goal-witness cache, which is the solver measured in most archived runs, and **FAST2** for the complete framework.

For every state  $s$ , FAST2 keeps:

- $F(s)$ , the truncated frontier with full vectors, stored either as a contiguous array of  $M$ -dimensional rows or as a dynamic k-d tree over  $\text{Tr}$  (Shperberg et al. 2026). Tree nodes live in one contiguous pool per frontier and are addressed by 32-bit indices. Every node stores the corners  $\text{lo}$  and  $\text{hi}$  of its subtree, removed members become tombstones, and the tree is rebuilt from its live members when its node count outgrows its size at the last build by a fixed factor.
- $m(s)$  and  $\mu(s)$ : the exact maximum first coordinate over the live members of  $F(s)$ , and its multiplicity.
- $G_{\text{cl}}(s)$ : all expanded path costs, as one contiguous array.
- $I(s)$ , built on the first heuristic selection at  $s$ :  $\text{Tr}(H(s))$  as one contiguous array; redundancy pointers  $\rho_s(i) = \max\{\ell < i : \text{Tr}(h^\ell) \preceq \text{Tr}(h^i)\}$  (0 if none); witness slots  $W_s(i)$ , initially undefined; and, if  $k(s) \geq \theta_H$ , a static k-d index  $K(s)$  over  $\text{Tr}(H(s))$ .

The goal frontier  $T \equiv F(s_{\text{goal}})$  uses the same adaptive representation.

#### 3.2 Algorithms

Algorithm 1 is the search loop. It differs from Algorithm 2 of Wolff et al. (2026) only in the order of the two tests applied to a generated successor (lines 16–17). The reference implementation initializes the start node with  $h = 0$  instead of  $h^1(s_{\text{start}})$ ; the start node is the only node in OPEN at the first extraction and is never re-inserted, so both initializations yield the same trajectory.

#### Algorithm 1 FAST2-SEARCH

Input : MOS instance  $P = \langle S, E, c, s\_start, s\_goal \rangle$ ;  
admissible MVH  $H$ , every  $H(s) = \langle h^1, \dots, h^k(s) \rangle$  stored in non-decreasing  $\leq_{lex}$  order  
Output: cost-unique Pareto-optimal solution set  $Sols$

```

1:  $Sols \leftarrow \emptyset$ ;  $T \equiv F(s\_goal)$ 
2: for all  $s \in S$  do  $F(s) \leftarrow \emptyset$ ;  $G\_cl(s) \leftarrow \emptyset$ ;  $m(s) \leftarrow 0$ ;  $\mu(s) \leftarrow 0$ 
3:  $n \leftarrow$  node with  $s(n) = s\_start$ ,  $g(n) = 0$ ,  $i(n) = 1$ ,  $f(n) = h^1(s\_start)$ 
4:  $OPEN \leftarrow \{n\}$   $\triangleright$  ordered by  $\leq_{lex}$  on  $f$ 
5: while  $OPEN \neq \emptyset$  do
6:    $n \leftarrow OPEN.EXTRACT-MIN()$ 
7:   if  $GOAL-DOM(f(n))$  then  $\triangleright$  lazy re-evaluation
8:      $i \leftarrow CHOOSE-H(s(n), g(n), i(n) + 1, true)$ 
9:     if  $i \neq \perp$  then  $INSERT(s(n), g(n), i)$ 
10:    continue
11:   if  $LOCAL-DOM(s(n), g(n))$  then continue
12:    $FRONTIER-UPDATE(s(n), g(n))$ ;  $G\_cl(s(n)) \leftarrow G\_cl(s(n)) \cup \{g(n)\}$ 
13:   if  $s(n) = s\_goal$  then  $Sols \leftarrow Sols \cup \{n\}$ ; continue
14:   for all  $s' \in Succ(s(n))$  do
15:      $g' \leftarrow g(n) + c(s(n), s')$ 
16:     if  $LOCAL-DOM(s', g')$  then continue  $\triangleright$  local test before selection
17:      $i' \leftarrow CHOOSE-H(s', g', 1, false)$ 
18:     if  $i' \neq \perp$  then  $INSERT(s', g', i')$ 
19: return  $Sols$ 

INSERT( $s, g, i$ ): add to  $OPEN$  a node with state  $s$ , path cost  $g$ , index  $i$ ,  $f = g + h^i(s)$ 

```

Algorithm 2 contains both dominance tests. The goal test is a truncated orthant-emptiness query against  $T$ . The local test is exact (Lemma 1): it returns *false* when no Tr-dominator exists, *true* when a full-dimensional witness is found, and falls back to  $G_{cl}(s)$  otherwise.

#### Algorithm 2 FAST2-DOM-CHECK

Invariants:  $F(s)$  holds every  $g' \in G_{cl}(s)$  whose truncation is not weakly dominated  
by the truncation of a later expansion at  $s$ ;  $m(s) = \max\{p_1 : p \in F(s)\}$

```

1: function  $GOAL-DOM(f)$   $\triangleright$  goal test (truncated)
2:   return  $EXISTS(T, Tr(f))$ 
3: function  $LOCAL-DOM(s, g)$   $\triangleright$  local test (exact)
4:    $\tau \leftarrow WITNESS(s, g)$ 
5:   if  $\tau = NONE$  then return false  $\triangleright$  no Tr-dominator, hence no dominator
6:   if  $\tau = FULL$  then return true  $\triangleright$  full-dimensional witness
7:   return  $\exists g' \in G_{cl}(s) : g' \leq g$   $\triangleright$  exact fallback
8: function  $WITNESS(s, g)$ 
9:   if  $m(s) \leq g_1$  then  $\triangleright$  monotonicity certificate
10:    return FULL if  $EXISTS(F(s), Tr(g))$  else NONE
11:    $\tau \leftarrow NONE$ 
12:   for all  $p \in F(s)$  with  $Tr(p) \leq Tr(g)$  do  $\triangleright$  corner-pruned orthant enumeration
13:     if  $p_1 \leq g_1$  then return FULL
14:    $\tau \leftarrow TR$ 
15:   return  $\tau$ 
16: function  $EXISTS(F, q)$   $\triangleright$  orthant-emptiness query
17:   return  $\exists p \in F : Tr(p) \leq q$   $\triangleright$  the first hit ends the query

```

Algorithm 3 selects the heuristic vector. It returns the first index at or after  $j$  whose truncated evaluation is not dominated by  $T$ , exactly as  $CHOOSEH$  does, but it answers most refutations without querying  $T$ .

Algorithm 3 FAST2-CHOOSE-H( $s, g, j, \text{reeval}$ )

Input : state  $s$ ; path cost  $g$ ; first candidate index  $j$ ;  
 $\text{reeval} = \text{true}$  iff  $h^{j-1}$  has just been refuted by GOAL-DOM

Output:  $\min\{ i \geq j : \neg \text{EXISTS}(T, \text{Tr}(g) + \text{Tr}(h^i)) \}$ , or  $\perp$  if there is none

```

1: if  $j > k(s)$  then return  $\perp$ 
2: if  $T = \emptyset$  then return  $j$ 
3: if  $k(s) \geq \theta_H$  then return KD-SELECT( $s, g, j$ )      ▷ optional static index
4:  $r \leftarrow j - 1$  if  $\text{reeval}$  else  $j$                   ▷ indices  $r, \dots, i - 1$  are refuted
5:  $w \leftarrow \text{undefined}$                                ▷ last goal witness found in this call
6: for  $i \leftarrow j$  to  $k(s)$  do
7:   if  $p_s(i) \geq r$  then continue                      ▷  $\text{Tr}(h^p(i)) \leq \text{Tr}(h^i)$  and  $h^p(i)$  is refuted
8:    $q \leftarrow \text{Tr}(g) + \text{Tr}(h^i)$ 
9:   if  $W_s(i) \leq q$  or  $w \leq q$  then continue          ▷ a cached goal witness refutes  $h^i$ 
10:   $t \leftarrow$  a member of  $T$  with  $\text{Tr}(t) \leq q$ , or  $\perp$  if none
11:  if  $t = \perp$  then return  $i$                           ▷ certified survivor
12:   $W_s(i) \leftarrow \text{Tr}(t)$ ;  $w \leftarrow \text{Tr}(t)$ 
13: return  $\perp$ 

```

Procedure 4 maintains  $F(s)$ , its representation, and  $m(s)$ .  $[\cdot]$  denotes the Iverson bracket.

Procedure 4 FRONTIER-UPDATE( $s, g$ )

Parameters: promotion size  $\theta_n$ ; promotion scan cost  $\theta_c$ ; rebuild factors  $\alpha, \beta$

```

1:  $D \leftarrow \{ p \in F(s) : \text{Tr}(g) \leq \text{Tr}(p) \}$       ▷ upper-orthant sweep, hi-corner pruned
2:  $F(s) \leftarrow (F(s) \setminus D) \cup \{g\}$               ▷ array: compaction; tree: tombstones, leaf
insertion
3: if  $g_1 > m(s)$  then  $(m(s), \mu(s)) \leftarrow (g_1, 1)$   ▷ exact monotonicity tracking
4: else
5:    $\mu(s) \leftarrow \mu(s) + [g_1 = m(s)] - |\{ p \in D : p_1 = m(s) \}|$ 
6:   if  $\mu(s) = 0$  then  $(m(s), \mu(s)) \leftarrow$  maximum of  $p_1$  over  $F(s)$  and its multiplicity
7: if  $F(s)$  is an array,  $|F(s)| \geq \theta_n$ , and its recent queries cost  $\geq \theta_c$  comparisons on average then
8:   replace the array by a median-split k-d tree over  $\text{Tr}(F(s))$ 
9: else if  $F(s)$  is a tree with more than  $\alpha \cdot b(s) + \beta$  nodes,  $b(s)$  its size at the last build, then
10:  rebuild the tree from its live members

```

Procedure 5 is the optional static index. It extends KD-ChooseH (Anonymous 2026a) with a fast path and with index-range pruning, so that it returns the minimum surviving index without classifying every member.

Procedure 5 KD-SELECT( $s, g, j$ )

Node data: corners  $N.\min \leq \text{Tr}(h^i) \leq N.\max$  for every member  $h^i$  of  $N$ ;  
lowest and highest member index  $N.\text{lo}, N.\text{hi}$ ; sorted member indices

```

1: if  $\neg \text{EXISTS}(T, \text{Tr}(g) + \text{Tr}(h^j))$  then return  $j$     ▷ fast path
2:  $b \leftarrow \infty$ 
3: for all  $N \in K(s)$  in depth-first order, children in increasing  $N.\text{lo}$  do
4:   if  $N.\text{hi} \leq j$  or  $N.\text{lo} \geq b$  then skip  $N$           ▷ index-range pruning
5:   else if  $\text{EXISTS}(T, \text{Tr}(g) + N.\min)$  then skip  $N$     ▷ Rule 1: every member refuted
6:   else if  $\neg \text{EXISTS}(T, \text{Tr}(g) + N.\max)$  then          ▷ Rule 2: no member refuted
7:      $b \leftarrow \min(b, \min\{ i \in N : i > j \})$ ; skip  $N$ 
8:   else if  $N$  is a leaf then
9:      $b \leftarrow \min(b, \min\{ i \in N : j < i < b, \neg \text{EXISTS}(T, \text{Tr}(g) + \text{Tr}(h^i)) \})$ 
10: return  $b$  if  $b < \infty$  else  $\perp$ 

```

### 3.3 Architectural pillars

**Pillar 1: dynamic frontier indexing.** Every  $F(s)$ , including  $T$ , starts as a contiguous array. It is promoted to a k-d tree (Shperberg et al. 2026) once it holds at least  $\theta_n = 8$  vectors and its recent queries cost at least  $\theta_c = 64$  comparisons on average; the running mean is halved every 4,096 queries so that it follows recent behaviour. The tree answers EXISTS and WITNESS by depth-first search that prunes a subtree when its corner minimum lo does not weakly dominate the query, and it performs the update sweep with corner-maximum pruning. Between rebuilds the tree is not rebalanced, so the worst-case query bound of balanced k-d trees applies only after a rebuild; Shperberg et al. (2026) nevertheless measured sublinear query costs on search-generated streams. The cost model replaces a dimension-specific size threshold. At  $M = 3$  tree queries cost about the same as array scans while tree updates cost more, and no state is promoted; at  $M \geq 4$  promotion happens early (Section 4.5). This matches the separation of Shperberg et al. (2026): a total-order scan is optimal up to a constant factor when  $d = 2$  and degrades to  $n$  tests when  $d \geq 3$ .

**Pillar 2: static heuristic indexing.**  $K(s)$  indexes  $\text{Tr}(H(s))$  with aggregate corners (Procedure 5). It is built only for states with  $k(s) \geq \theta_H$ , and  $\theta_H = \infty$  by default. Section 4 explains why the index pays in the baseline’s operation order but not in FAST2’s.

**Pillar 3: architectural and cache optimizations.**

- *Operation reordering.* At generation, LOCAL-DOM precedes CHOOSE-H (Algorithm 1, lines 16–17). Both are pure predicates, so the inserted node is unchanged (Lemma 6), but locally dominated successors, which are the majority, are discarded before any heuristic vector is tested. Reordering removes 67–80% of the selection calls (Section 4.4).
- *Memory locality.* Every per-state set is a contiguous array of  $M$ -dimensional rows:  $F(s)$  before promotion,  $G_{cl}(s)$ , and  $\text{Tr}(H(s))$ . The reference implementation keeps the same sets as vectors of vectors and closed paths as shared node pointers. Trees are allocated only for promoted states.
- *Witness capture and caching.* The local test enumerates Tr-dominators and stops at the first one that also satisfies  $p_1 \leq g_1$  (Algorithm 2, line 13). A full-dimensional witness found during the truncated scan settles the test without the  $G_{cl}(s)$  fallback. In CHOOSE-H, the goal vector that refuted  $h^i$  at  $s$  is cached in  $W_s(i)$  and tested first at the next call; the witness of the preceding candidate is tested second (Algorithm 3, line 9). Paths that reach the same state have similar costs, so a cached vector usually refutes again with a single comparison.
- *Exact monotonicity tracking.*  $m(s)$  is maintained exactly with a multiplicity counter (Procedure 4, lines 3–6), at constant cost per update and with a recomputation only when the last live vector attaining the maximum is removed. Lemma 1 requires  $m(s) \geq \max\{p_1 : p \in F(s)\}$ . A stale, too-high value is sound but weakens the certificate: the earlier frontier-tree solvers, which recomputed  $m(s)$  only at rebuilds, performed 3–12% more fallbacks than the reference implementation on the landmark grids of Table 9. A value taken from the most recently inserted vector is unsound under MVHs (Section 2.3). With exact  $m(s)$ , the certificate (Algorithm 2, line 9) is the special case of the witness scan in which the first Tr-dominator found is already a full witness; the implementation fuses lines 9–15 into one traversal.
- *Redundancy pointers.*  $\rho_s(i)$  is the latest earlier vector whose truncation weakly dominates  $\text{Tr}(h^i)$ . If that vector has been refuted in the current call, or by the goal test that triggered a re-evaluation,  $h^i$  is refuted without a query (Algorithm 3, line 7). Landmark families are 65–95% redundant in this sense. In an antichain stored in non-decreasing order of  $h_1$ , no earlier member Tr-dominates a later one (Section 4.2), so the pointers are almost always empty on A\*pex MVHs; the quadratic construction is skipped when a 48-vector prefix of  $H(s)$  contains no redundancy, which is always sound.

### 3.4 Correctness

**Lemma 3 (Redundancy skip).** If  $\rho_s(i) \geq r$  at line 7 of Algorithm 3, then  $h^i$  is refuted.

*Proof.*  $\ell = \rho_s(i)$  lies in  $[r, i)$ . If  $\ell \geq j$ ,  $h^\ell$  was refuted earlier in the same call (by line 7 or 9, inductively, or by lines 10–12). If  $\ell = j - 1$ , the call is a re-evaluation and  $h^\ell$  was refuted by GOAL-DOM in the same iteration, with  $T$  unchanged since. Hence some  $t \in T$  satisfies  $\text{Tr}(t) \preceq \text{Tr}(g) + \text{Tr}(h^\ell) \preceq \text{Tr}(g) + \text{Tr}(h^i)$ .  $\square$

**Lemma 4 (Goal-witness cache).** If  $W_s(i) \preceq q$  or  $w \preceq q$  at line 9 of Algorithm 3, then  $\text{Exists}(T, q)$ .

*Proof.* Both vectors are truncations of vectors that were members of  $T$ ; apply Lemma 2. A miss falls through to the exact query.  $\square$

**Lemma 5 (Static index).** For  $j \leq k(s)$  and  $T \neq \emptyset$ ,  $\text{KD-SELECT}(s, g, j)$  returns the same index as  $\text{CHOOSE-H}(s, g, j)$ .

*Proof.* Rule 1 skips only refuted members, because  $\text{Tr}(g) + N.\min \preceq \text{Tr}(g) + \text{Tr}(h^i)$ . Rule 2 accepts only survivors, because  $\text{Tr}(g) + \text{Tr}(h^i) \preceq \text{Tr}(g) + N.\max$ . Index-range pruning discards only indices not greater than  $j$ , which the fast path has settled, or not smaller than a known survivor. The returned  $b$  is therefore the minimum surviving index.  $\square$

**Lemma 6 (Reordering).** For every generated pair  $(s', g')$ , Algorithm 1 inserts a node if and only if the reference algorithm does, and with the same index.

*Proof.* CHOOSE-H reads  $T$  and writes only  $W$ ,  $w$ , and the lazily built index  $I(s)$ , none of which changes an answer (Lemma 4). LOCAL-DOM reads  $F$ ,  $G_{\text{cl}}$ , and  $m$ , and writes nothing. Neither test modifies what the other reads, so the conjunction  $\neg \text{Local} - \text{Dom} \wedge \text{Choose} - H \neq \perp$  has the same value in either order, and the selected index depends only on  $(s', g', T)$ .  $\square$

**Theorem 2 (Trajectory equivalence).** Given the same MVH in the same stored order and the same OPEN implementation, FAST2 and L-NAMOA\*dr-mvh perform identical sequences of OPEN insertions and extractions. They return the same solutions in the same order and perform the same numbers of expansions, extractions, and re-insertions.

*Proof.* By induction over iterations, both algorithms hold the same OPEN and the same live frontier sets. GOAL-DOM evaluates the reference goal test over the same live set: arrays and trees answer the same existence predicate, and lazy deletion is exact (Shperberg et al. 2026, Proposition 1, whose proof does not require the stored set to be an antichain, as  $F(s)$  need not be after truncation). LOCAL-DOM equals the reference local test, because both equal  $[\exists g' \in G_{\text{cl}}(s) : g' \preceq g]$  (Lemma 1; its proof applies to WITNESS verbatim). CHOOSE-H returns the same index by Lemmas 3–5. Generation inserts the same nodes in the same successor order by Lemma 6. FRONTIER-UPDATE maintains the same live set as the reference update: remove every vector whose truncation is weakly dominated by  $\text{Tr}(g)$ , then insert  $g$ .  $\square$

**Corollary 1.** If every  $H(s)$  is stored in non-decreasing  $\leq_{\text{lex}}$  order, FAST2 is complete and sound (Theorem 1; Wolff et al. 2026, Theorem 2).

Ties among equal  $f$ -vectors are broken by the priority-queue implementation. Trajectory identity therefore holds between executables built against the same standard library, which is the case for every comparison in Section 5.

---

## 4 Architectural Analysis of Heuristic Indexing

---

Static and dynamic k-d trees index the same kind of object, a set of truncated vectors, yet the dynamic frontier trees of Pillar 1 are FAST2’s largest single source of speedup at  $M \geq 4$ , while the static heuristic trees of Pillar 2 do not pay in FAST2’s operation order. This section explains the difference.

## 4.1 Certification and refutation

**Proposition 5 (Certification).** Let an exact CHOOSE-H observe  $T$  only through the predicate  $\text{Exists}(T, \cdot)$  (in particular, not through its size). Whenever it returns  $i \neq \perp$ , it has received the answer *false* for some query  $q$  with  $\text{Tr}(g) + \text{Tr}(h^i) \preceq q$ .

*Proof.* Suppose not, and add to  $T$  a vector  $t'$  with  $\text{Tr}(t') = \text{Tr}(g) + \text{Tr}(h^i)$ . Positive answers stay positive. A negative answer for  $q$  would become positive only if  $\text{Tr}(g) + \text{Tr}(h^i) \preceq q$ , which no negative query satisfies by assumption. The procedure receives the same answers and returns  $i$ , although  $h^i$  is now refuted.  $\square$

A negative EXISTS query cannot stop early: it must exhaust every part of the frontier index that the query orthant meets, which is the whole array in a linear representation. Every successful selection on a non-empty goal frontier therefore pays at least one negative query, whatever index is kept over  $H(s)$ ; Algorithm 3 skips all queries only while  $T = \emptyset$ . An index over  $H(s)$  can save only refutations, and only when one aggregate test replaces several member tests. With a linear scan, a successful call pays one positive query per skipped candidate plus one negative query, and a call that returns  $\perp$  pays positive queries only.

In FAST2's operation order the first candidate is accepted in 86–93% of the selections on landmark grids with  $M \leq 5$ , in 73% at  $M = 6$ , and in 21% on the A\*pex grid (Table 5). Most of the remaining refutations are answered without querying  $T$ : by redundancy pointers on landmark MVHs, and by the goal-witness cache, which answered 77–91% of the goal queries on road networks (Table 14). Little is left for a heuristic tree to save.

## 4.2 Box tightness on anti-correlated heuristic sets

Let  $O(x) = \{y \in \mathbb{R}^d : y \preceq x\}$  denote the lower orthant of  $x$ , and  $x_h = \text{Tr}(g) + \text{Tr}(h)$  the truncated evaluation of  $h$ . A member  $h$  is refuted iff  $T$  meets  $O(x_h)$ .

**Proposition 6 (Tightness of aggregate tests).** For every node  $N$  of  $K(s)$ ,

$$O(\text{Tr}(g) + N.\text{min}) = \bigcap_{h \in N} O(x_h), \quad O(\text{Tr}(g) + N.\text{max}) \supseteq \bigcup_{h \in N} O(x_h).$$

Rule 1 fires exactly when all members of  $N$  are refuted, for every  $T$ , if and only if  $N$  has a least member under  $\preceq$  on truncations. Rule 2 fires exactly when no member is refuted, for every  $T$ , if and only if  $N$  has a greatest member.

*Proof.* A vector is weakly dominated by every  $x_h$  iff it is weakly dominated by their component-wise minimum  $\text{Tr}(g) + N.\text{min}$ , which gives the identity; the inclusion follows from  $x_h \preceq \text{Tr}(g) + N.\text{max}$ . If  $N$  has a least member  $h_0$ , then  $N.\text{min} = \text{Tr}(h_0)$  and refuting  $h_0$  refutes every member. If it has none, let  $T = \{x_h : h \in N\}$ : every member is refuted, but some  $x_h \preceq \text{Tr}(g) + N.\text{min}$  would make  $h$  a least member, so Rule 1 does not fire. The argument for Rule 2 is symmetric, with  $T = \{\text{Tr}(g) + N.\text{max}\}$ .  $\square$

For example, with  $\text{Tr}(H_N) = \{(0, 10), (5, 5), (10, 0)\}$  the corners are  $(0, 0)$  and  $(10, 10)$ ;  $T = \{x_h : h \in N\}$  refutes every member while Rule 1 fails, and  $T = \{\text{Tr}(g) + (6, 6)\}$  refutes no member while Rule 2 fails. Three observations connect Proposition 6 to the heuristic sets of our benchmarks.

1. **Truncation reorients dominance.** If  $H(s)$  is an antichain stored in non-decreasing order of  $h_1$ , no member Tr-dominates a later member:  $\text{Tr}(h^i) \preceq \text{Tr}(h^j)$  with  $i < j$  would give  $h^i \preceq h^j$ . Every truncated dominance inside such a set therefore points backward, from a later member (larger  $h_1$ , smaller truncation) to an earlier one. Redundancy pointers, which serve a forward scan, stay empty, whereas a spatial index can exploit these relations through its aggregate tests. The archived A\*pex heuristics contain many of them: 83–96% of the members on the grids and 2.5–21% on the New York subgraphs (growing as  $\varepsilon$  decreases) are Tr-dominated by another member of the same set. This is why the static index pays on the A\*pex grids when scans are long (Table 6).

2. **Acceptance almost never fires.** Rule 2 is the only rule that shortens certification, and it can fire only when every member of the subtree survives. Selection is expensive exactly when many candidates are refuted, and then wholly surviving subtrees are rare. On nyr3k ( $M = 5$ ,  $\varepsilon = 0.01$ ,  $\max |H(s)| = 2,839$ , leaf size 8) Rule 1 fired 163,566 times and Rule 2 fired 78 times, over 331,630 internal recursions, 119,840 leaf visits, and 625,118 member tests; every one of these tests is a frontier query, and a failed Rule 1 test is a negative one (Proposition 5). The index took 0.86–0.87 s against 0.68 s for the linear scan.
3. **Landmark families and single-valued bounds.** Landmark files are unsorted, and 65–95% of their vectors are Tr-dominated by an *earlier* member of the same set. These forward relations are exactly what redundancy pointers exploit at constant cost per skip, and what maximum aggregation removes altogether (Proposition 1). A single-valued heuristic is the limiting case of one vector: its box is the vector itself, and both rules reduce to the member test.

#### 4.3 Frontier trees versus heuristic trees

The two indices differ in what a node test costs (Table 4).

**Table 4.** Dynamic frontier index versus static heuristic index.

|                            | dynamic frontier index $(F(s), T)$                                             | static heuristic index $K(s)$                                                          |
|----------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| indexed set                | truncated path costs                                                           | truncated heuristic vectors                                                            |
| query                      | point against set: $\text{Exists}(F, q)$                                       | set against set: first $i$ with $\neg \text{Exists}(T, x_{hi})$                        |
| node test                  | $\text{lo}(\nu) \preceq q$ : $d$ comparisons                                   | $\text{Exists}(T, \cdot)$ : a full frontier query                                      |
| cost of a failed prune     | $d$ comparisons                                                                | one negative frontier query                                                            |
| exactness of the node test | $\text{lo}(\nu) \preceq q$ iff the box of $\nu$ meets $O(q)$                   | exact iff the subtree has a least (Rule 1) or greatest (Rule 2) member (Proposition 6) |
| worst-case query           | $O(d n^{1-1/d})$ on a balanced tree (Lee and Wong 1977; Shperberg et al. 2026) | no sublinear bound; one certification per call (Proposition 5)                         |
| construction               | incremental, tombstones, amortized rebuild                                     | static, once per state; 7–9% of run time on A*pex grids                                |

A frontier tree is a tree of comparisons; a heuristic tree is a tree of oracle calls, each as expensive as the scan step it is meant to save. A frontier tree pays when one box test excludes many stored vectors for the price of  $d$  comparisons. A heuristic tree pays only when one aggregate test excludes many heuristic vectors for the price of one frontier query that the scan would pay once per vector. That condition requires long refutation runs over groups whose boxes are tight. Reordering removes most long runs (Section 4.4), and the accept rule, the only one that could shorten certification, almost never fires (Section 4.2).

The representation of  $T$  modulates the cost of failed aggregate tests, because corner pruning shortens negative queries. In the baseline order on the  $M = 4$  A\*pex grid, the static index was 1.83× faster than the linear scan when  $T$  was a k-d tree, but 0.62× as fast when  $T$  was an array (archived sweep E3b).

#### 4.4 Reordering shifts the computational profile

Table 5 compares the two operation orders with every other FAST mechanism fixed (archived ablation E4, first repetition; outcome shares are over all selection calls, scan depth is the mean distance between the returned and the first candidate index).

**Table 5.** Heuristic selection under the baseline order (CHOOSE-H first) and FAST’s order (local test first).

| instance                        | order    | selection calls at generation | goal queries by selection | first candidate accepted | $\perp$ | mean scan depth | selection time | local-test time |
|---------------------------------|----------|-------------------------------|---------------------------|--------------------------|---------|-----------------|----------------|-----------------|
| grid 15×15, M=3, landmark K=200 | baseline | 1,233,536                     | 1,638,092                 | 86.5%                    | 7.6%    | 0.80            | 14.7%          | 17.1%           |
|                                 | FAST     | 333,024                       | 370,208                   | 92.8%                    | 4.0%    | 0.35            | 4.5%           | 19.9%           |
| grid 10×10, M=4, landmark K=50  | baseline | 1,243,670                     | 2,969,217                 | 69.2%                    | 19.3%   | 0.92            | 22.2%          | 22.4%           |
|                                 | FAST     | 297,332                       | 435,367                   | 85.8%                    | 8.9%    | 0.50            | 4.3%           | 29.9%           |
| grid 12×12, M=4, landmark K=50  | baseline | 1,442,658                     | 2,819,118                 | 70.7%                    | 16.3%   | 1.41            | 22.2%          | 23.3%           |
|                                 | FAST     | 331,275                       | 418,301                   | 88.7%                    | 7.5%    | 0.43            | 4.3%           | 30.7%           |
| grid 10×10, M=5, landmark K=50  | baseline | 2,775,914                     | 5,954,513                 | 71.5%                    | 20.9%   | 0.68            | 25.3%          | 22.3%           |
|                                 | FAST     | 708,471                       | 912,555                   | 88.6%                    | 8.6%    | 0.19            | 5.1%           | 30.3%           |
| grid 10×10, M=6, landmark K=25  | baseline | 1,241,154                     | 3,983,734                 | 44.7%                    | 21.7%   | 2.03            | 37.8%          | 27.6%           |
|                                 | FAST     | 249,364                       | 365,790                   | 72.5%                    | 3.7%    | 0.53            | 7.3%           | 43.0%           |
| grid 10×10, M=4, A*pex          | baseline | 252,312                       | 31,043,684                | 8.8%                     | 67.2%   | 53.50           | 82.4%          | 4.9%            |
|                                 | FAST     | 84,512                        | 8,440,670                 | 21.1%                    | 30.7%   | 37.59           | 64.5%          | 13.5%           |

Reordering reduces the selection calls at generation by 3.0–5.0× and the goal queries they issue by 3.7–10.9×. The share of calls that end in  $\perp$  falls from 8–22% to 4–9% on landmark MVHs and from 67% to 31% on the A\*pex MVH; the first candidate is accepted more often; and the mean scan depth falls by 30–74%. Selection falls from 15–38% to 4–7% of the running time on landmark grids and from 82% to 65% on the A\*pex grid, and the local test becomes the largest single cost on landmark grids (20–43%).

The mechanism is direct. A successor discarded by the local test is weakly dominated at its state by an expanded path. Its cost vector is large, so its candidates are refuted by  $T$  for many indices, and its scan often ends at  $\perp$ . The baseline pays this scan before it discovers that the path is dominated. After reordering, CHOOSE-H sees only locally non-dominated paths, whose first candidate usually survives. The principle is that of partial-expansion A\* (Felner et al. 2012), which avoids paying for surplus nodes that will never be expanded; here the surplus work is heuristic selection for paths that the local test discards. The static heuristic index was designed to shorten long refutation runs, and reordering removes most of them (Table 6). Building the index alone takes 7–9% of the running time on A\*pex grids; with  $\theta_H = 1024$  on the grids of the ablation, the index configuration took 0.92–1.16 times FAST’s time.

**Table 6.** Linear scan versus the best static-index configuration on the two A\*pex grids (archived sweeps E3c and P2; minimum of three runs; all outputs identical).

| operation order           | instance        | linear scan (s) | best static index (s) | configuration             | ratio |
|---------------------------|-----------------|-----------------|-----------------------|---------------------------|-------|
| baseline (CHOOSE-H first) | grid 10×10, M=3 | 0.096           | 0.075                 | every state, leaf size 32 | 1.28× |
| baseline (CHOOSE-H first) | grid 10×10, M=4 | 1.427           | 0.754                 | every state, leaf size 8  | 1.89× |
| FAST (local test first)   | grid 10×10, M=3 | 0.047           | 0.048                 | $\theta_H = 3000$         | 0.98× |
| FAST (local test first)   | grid 10×10, M=4 | 0.640           | 0.580                 | $\theta_H = 1000$         | 1.10× |

#### 4.5 Pointer indirection versus contiguous iteration

**Layout alone.** The reference algorithm on contiguous storage, with no trees and no other change, is 1.46× (M = 3), 2.68–3.43× (M = 4), 3.40× (M = 5), 3.16× (M = 6), and 1.70× (A\*pex grid) faster than the reference implementation, with identical output (Table 10). The reference implementation reaches frontier members through vectors of vectors and closed paths through shared node pointers.

**Crossover.** Table 7 reports the archived per-operation costs of arrays and k-d trees by frontier size. At  $M = 3$  local frontiers stay below 64 vectors, query costs are at parity, and updates cost 1.4–3.1× more in a tree; building trees at every state makes the search 0.85–0.89× as fast (archived sweeps E1 and E2). At  $M = 4$  the local-query crossover lies between 64 and 256 vectors, and at  $M = 5$  near 32–64. Goal queries cross over at 8–32 vectors on landmark grids with  $M \geq 4$ , but only at 512–1,024 vectors on the A\*pex grid, where most goal queries are positive and an array scan exits early.

**Table 7.** Cycles per operation by frontier size (archived per-size profile; rdtsc per operation).

| operation   | M | MVH      | frontier size | array  | k-d tree |
|-------------|---|----------|---------------|--------|----------|
| local query | 3 | landmark | 32–63         | 266    | 248      |
| local query | 4 | landmark | 64–127        | 454    | 463      |
| local query | 4 | landmark | 512–1,023     | 1,876  | 635      |
| local query | 5 | landmark | 32–63         | 396    | 325      |
| local query | 5 | landmark | 2,048–4,095   | 10,113 | 1,035    |
| goal query  | 4 | landmark | 16–31         | 101    | 65       |
| goal query  | 5 | landmark | 2,048–4,095   | 4,132  | 307      |
| goal query  | 4 | A*pex    | 256–511       | 104    | 121      |
| goal query  | 4 | A*pex    | 1,024–2,047   | 225    | 140      |
| update      | 3 | landmark | 8–15          | 404    | 1,269    |
| update      | 4 | landmark | 64–127        | 1,408  | 1,865    |
| update      | 5 | landmark | 2,048–4,095   | 24,182 | 3,583    |

**Promotion.** The cost model never promotes at  $M = 3$ . It promotes 31–49 of the 100–144 states at  $M = 4$ , 44–46 at  $M = 5$  and 6, and 10 on the A\*pex grid. It is within noise of the best fixed size threshold (a fixed threshold of 16 took 0.94–1.12 times FAST’s time) and avoids the  $M = 3$  regression. Removing trees altogether costs 1.02× at  $M = 3$  and up to 6.31× at  $M = 6$  (Table 15).

**Node layout.** Padding k-d nodes to ten coordinates instead of  $M$  made no measurable difference (0.945 s against 1.088 s at  $M = 4$ ; 3.89 s against 3.86 s at  $M = 5$ ). The locality effect lies at the container level, in the indirection through vectors of vectors and node pointers, not within a node.

We attribute the array’s advantage below the crossover to unit-stride access with early exit, which hardware prefetching serves well, and the tree’s overhead to data-dependent traversal of an index-linked node pool. We did not measure cache misses; the attribution rests on the layout-only control and on the per-size crossover.

## 5 Empirical Survey and Benchmark Synthesis

This section consolidates every archived run of the project; no search was re-run. Consolidated tables were regenerated from the archived CSV and log files by a script (Appendix A), not transcribed by hand.

### 5.1 Protocol

All runs used one laptop (Intel Core i5-1135G7, 4 cores, 1.25 MB L2 per core, 8 MB L3, 15.7 GB RAM, Windows) and one toolchain: clang 21.1 with libc++, -std=c++20 -O2 -DNDEBUG -march=native. The reference implementation of L-NAMOA\*dr-mvh (repository bridging-mvh-dr, commit 0a2f9ea) was compiled unmodified into the same executables as the solvers it was compared with. Each process ran at high priority on a pinned core, configurations were interleaved, and the minimum over repetitions is reported: three repetitions unless stated otherwise, one for the baseline. Run-to-run variation on this machine reaches 2× under thermal throttling; differences below about 10% are not significant.

The solvers are:

- **BASE**: the reference L-NAMOA\*dr-mvh (Wolff et al. 2026).
- **SKD**: BASE with the dynamic k-d frontier of Shperberg et al. (2026) (earlier in-house integration).
- **KDH**: BASE with the static heuristic index of KD-ChooseH (Anonymous 2026a) (earlier in-house integration).
- **DUAL**: SKD and KDH combined (earlier in-house integration).
- **FAST**: FAST2 without the goal-witness cache; either the ported solver (Tables 9, 11–13) or its prototype (Tables 10, 14, 15).
- **FAST2**: the prototype with the goal-witness cache.

Default parameters are  $\theta_n = 8$ ,  $\theta_c = 64$ , and  $\theta_H = \infty$ . *exp* counts expansions. *gen* is the reference counter *num\_generation*, which is incremented at every extraction from OPEN, re-inserted nodes included. The archived identity protocol compared, on grids, the byte content of the solution files (solutions in discovery order) together with *exp* and *gen*; on road networks, the hash of the solution set together with *exp*. Section 5.7 adds an independent re-verification.

## 5.2 Benchmarks

**Table 8.** Instance families.  $|H(s)|$  gives the average and maximum over states.

| family          | graphs                                                                                                                                                                                                                                                        | M    | heuristic                                                                                         | $ H(s) $                 |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------------------------------------------------|--------------------------|
| land mark grids | 4-neighbour grids, 8×8 to 15×15, correlated costs ( $\rho \in \{-0.6, -0.3, 0, 0.3\}$ ), seed 42                                                                                                                                                              | 3–6  | $K \in \{1, \dots, 200\}$ landmark vectors, Dijkstra on the symmetrised search graph (admissible) | $K$ ; 65–95% redundant   |
| A*pex grids     | 10×10 grids                                                                                                                                                                                                                                                   | 3, 4 | A*pex MVH (builder of Wolff et al.)                                                               | 314 and 1,469 on average |
| New York (NY)   | BFS balls of 1,000–8,000 vertices around vertex 140000 of the 9th DIMACS New York graph (Demetrescu, Goldberg, and Johnson 2009); $c_1$ = distance, $c_2$ = travel time, $c_3, \dots$ uniform in $[1, 100]$ (seed 42); start = farthest vertex, goal = centre | 3–6  | A*pex MVH with $\varepsilon \in \{0, 0.01, 0.05, 0.1\}$ ; the solver is exact                     | 10–2,695 / 54–32,966     |
| legac y grids   | 8×8 to 25×25                                                                                                                                                                                                                                                  | 3–6  | landmark vectors from Dijkstra on the <i>directed</i> edges (inadmissible, Section 5.8)           | $\leq 100$               |

On road networks  $\varepsilon$  only shapes the heuristic: smaller  $\varepsilon$  yields larger and tighter sets  $H(s)$ , and the Pareto set is the same for every  $\varepsilon$  (Section 5.7). Nine heuristics, seven whose construction had exceeded 1,200 s with the original builder and two that the original sweep never reached, were built by a re-implementation of the A\*pex builder that makes the same decisions in the same order; it wrote byte-identical files on seven comparison instances and was 6–26× faster on them.

## 5.3 Grids

**Table 9.** Grids with admissible landmark MVHs; one executable, minimum wall time in seconds (BASE one run, KDH at  $M = 5$  one run, others three). All rows are identical. Fallbacks are full-dimensional local tests.

| grid                  | M | K       | $ T $      | exp         | gen         | BASE  | SKD   | KDH   | DUAL      | FAST      | BASE/FAS<br>T | DUAL/FAS<br>T | fallbacks BASE / SKD /<br>FAST |
|-----------------------|---|---------|------------|-------------|-------------|-------|-------|-------|-----------|-----------|---------------|---------------|--------------------------------|
| 10×10,<br>$\rho=-0.6$ | 3 | 50      | 1,602      | 35,862      | 67,241      | 0.166 | 0.305 | 0.211 | 0.28<br>4 | 0.08<br>8 | 1.88×         | 3.23×         | 19,217 / 21,442 / 6,999        |
| 15×15,<br>$\rho=-0.3$ | 3 | 10<br>0 | 2,178      | 139,01<br>0 | 261,61<br>8 | 1.150 | 1.676 | 1.081 | 1.32<br>5 | 0.42<br>7 | 2.69×         | 3.10×         | 27,262 / 29,252 / 3,101        |
| 15×15,<br>$\rho=-0.6$ | 3 | 20<br>0 | 2,776      | 170,05<br>2 | 338,22<br>7 | 1.387 | 2.852 | 1.246 | 1.61<br>4 | 0.73<br>1 | 1.90×         | 2.21×         | 19,498 / 21,258 / 2,825        |
| 10×10,<br>$\rho=-0.6$ | 4 | 10      | 12,66<br>7 | 185,39<br>4 | 291,13<br>9 | 5.035 | 3.302 | 6.748 | 3.52<br>3 | 0.99<br>5 | 5.06×         | 3.54×         | 46,972 / 51,635 / 1,804        |

| grid                  | M | K       | $ T $      | exp         | gen         | BASE       | SKD        | KDH        | DUAL      | FAST      | BASE/FAS<br>T | DUAL/FAS<br>T | fallbacks BASE / SKD /<br>FAST |
|-----------------------|---|---------|------------|-------------|-------------|------------|------------|------------|-----------|-----------|---------------|---------------|--------------------------------|
| 10×10,<br>$\rho=-0.6$ | 4 | 25      | 12,66<br>7 | 186,70<br>3 | 295,33<br>4 | 7.257      | 4.179      | 8.126      | 3.63<br>4 | 0.85<br>4 | 8.50×         | 4.25×         | 55,069 / 59,875 / 2,118        |
| 10×10,<br>$\rho=-0.6$ | 4 | 50      | 12,66<br>7 | 187,07<br>0 | 296,77<br>2 | 9.438      | 4.806      | 8.366      | 4.05<br>5 | 1.33<br>3 | 7.08×         | 3.04×         | 56,631 / 62,005 / 2,210        |
| 10×10,<br>$\rho=-0.6$ | 4 | 10<br>0 | 12,66<br>7 | 187,23<br>0 | 297,16<br>9 | 9.613      | 6.819      | 9.944      | 4.10<br>5 | 1.01<br>9 | 9.44×         | 4.03×         | 57,974 / 63,626 / 2,316        |
| 12×12, $\rho=0.0$     | 4 | 50      | 13,70<br>9 | 214,67<br>0 | 334,59<br>9 | 11.04<br>4 | 6.141      | 13.69<br>5 | 5.47<br>3 | 1.00<br>8 | 10.95×        | 5.43×         | 53,497 / 55,792 / 1,155        |
| 10×10,<br>$\rho=-0.3$ | 5 | 50      | 48,95<br>0 | 448,47<br>2 | 717,48<br>4 | 77.92<br>2 | 23.81<br>5 | 76.35<br>1 | –         | –         | –             | –             | 97,580 / 100,229 / –           |

**Table 10.** Prototype of FAST against the reference implementation (seconds). *Layout only* is the reference algorithm with contiguous frontier and closed sets and no other change; it and FAST are the same executable.

| grid               | M | heuristic       | $ T $  | exp     | gen     | BASE   | layout only | FAST  | BASE/layout | BASE/FAST |
|--------------------|---|-----------------|--------|---------|---------|--------|-------------|-------|-------------|-----------|
| 15×15, $\rho=-0.6$ | 3 | landmark, K=200 | 2,776  | 170,052 | 338,227 | 1.541  | 1.054       | 0.613 | 1.46×       | 2.51×     |
| 10×10, $\rho=-0.6$ | 4 | landmark, K=50  | 12,667 | 187,070 | 296,772 | 9.434  | 2.749       | 0.906 | 3.43×       | 10.41×    |
| 12×12, $\rho=0.0$  | 4 | landmark, K=50  | 13,709 | 214,670 | 334,599 | 11.225 | 4.194       | 1.063 | 2.68×       | 10.56×    |
| 10×10, $\rho=-0.3$ | 5 | landmark, K=50  | 48,950 | 448,472 | 717,484 | 87.370 | 25.714      | 3.348 | 3.40×       | 26.10×    |
| 10×10, $\rho=0.3$  | 6 | landmark, K=25  | 17,164 | 197,696 | 251,680 | 72.594 | 23.000      | 1.576 | 3.16×       | 46.07×    |
| 10×10              | 4 | A*pex           | 11,330 | 47,305  | 83,990  | 2.168  | 1.278       | 0.642 | 1.70×       | 3.37×     |

At  $M = 3$  FAST is 1.9–2.7× faster than BASE. None of this gain comes from frontier trees, since the cost model builds none; it comes from layout, witness capture, and reordering. The earlier integrations are slower than BASE at  $M = 3$  (SKD 0.49–0.69×, DUAL 0.58–0.87×), and KDH alone is at most 1.13× faster than BASE on any grid. At  $M = 4$  FAST is 5.1–11.0× faster than BASE and 3.0–5.4× faster than DUAL. Witness capture and exact tracking cut the full-dimensional fallbacks by 64–98% relative to BASE. The prototype is 26.1× faster at  $M = 5$ , 46.1× at  $M = 6$ , and 3.4× on the A\*pex grid; contiguous storage alone contributes 1.5–3.4×.

## 5.4 Road networks

**Table 11.** DIMACS New York subgraphs with genuine A\*pex MVHs (seconds). FAST2 was measured in the same session as BASE on the four formerly unbuildable instances only. All rows are identical.

| graph | M | $\epsilon$ | $ H(s) $ avg/max | $ T $  | exp     | gen     | BASE    | FAST  | FAST2 | BASE/FAST | BASE/FAST2 |
|-------|---|------------|------------------|--------|---------|---------|---------|-------|-------|-----------|------------|
| nyr3k | 4 | 0.1        | 10/54            | 646    | 12,692  | 34,915  | 0.148   | 0.056 | –     | 2.6×      | –          |
| nyr3k | 4 | 0.05       | 20/103           | 646    | 10,962  | 31,341  | 0.127   | 0.044 | –     | 2.9×      | –          |
| nyr3k | 4 | 0.01       | 91/555           | 646    | 8,929   | 25,725  | 0.191   | 0.050 | –     | 3.8×      | –          |
| nyr3k | 4 | 0          | 356/2,616        | 646    | 8,440   | 24,131  | 0.323   | 0.069 | –     | 4.7×      | –          |
| nyr3k | 5 | 0.1        | 24/157           | 4,313  | 55,894  | 153,580 | 12.242  | 0.497 | –     | 24.6×     | –          |
| nyr3k | 5 | 0.05       | 71/464           | 4,313  | 48,728  | 138,475 | 17.386  | 0.436 | –     | 39.9×     | –          |
| nyr3k | 5 | 0.01       | 457/2,839        | 4,313  | 42,321  | 119,629 | 37.773  | 0.620 | –     | 60.9×     | –          |
| nyr3k | 6 | 0.1        | 60/626           | 11,286 | 140,935 | 341,780 | 108.394 | 1.443 | –     | 75.1×     | –          |
| nyr3k | 6 | 0.05       | 194/2,250        | 11,286 | 127,349 | 315,007 | 134.200 | 1.505 | –     | 89.2×     | –          |
| nyr3k | 6 | 0.01       | 1,347/15,356     | 11,286 | 114,993 | 280,785 | 225.648 | 3.381 | 2.680 | 66.7×     | 84.2×      |
| nyr5k | 4 | 0.1        | 17/72            | 7,105  | 251,705 | 825,193 | 7.887   | 1.921 | –     | 4.1×      | –          |
| nyr5k | 4 | 0.05       | 42/200           | 7,105  | 216,370 | 748,525 | 8.189   | 1.982 | –     | 4.1×      | –          |
| nyr5k | 4 | 0.01       | 239/1,868        | 7,105  | 144,117 | 547,577 | 9.985   | 1.795 | –     | 5.6×      | –          |

| graph | M | $\epsilon$ | $ H(s) $ avg/max | $ T $  | exp     | gen       | BASE    | FAST   | FAST2 | BASE/FAST | BASE/FAST2 |
|-------|---|------------|------------------|--------|---------|-----------|---------|--------|-------|-----------|------------|
| nyr5k | 4 | 0          | 1,187/12,165     | 7,105  | 124,837 | 456,363   | 21.858  | 5.307  | 3.236 | 4.1×      | 6.8×       |
| nyr5k | 5 | 0.1        | 52/301           | 26,701 | 618,328 | 1,779,985 | 173.725 | 7.958  | –     | 21.8×     | –          |
| nyr5k | 5 | 0.05       | 172/1,472        | 26,701 | 503,505 | 1,575,427 | 192.466 | 9.168  | –     | 21.0×     | –          |
| nyr5k | 5 | 0.01       | 1,446/21,520     | 26,701 | 396,219 | 1,214,967 | 326.274 | 10.127 | 9.099 | 32.2×     | 35.9×      |
| nyr8k | 4 | 0.1        | 25/140           | 5,713  | 250,951 | 671,917   | 30.858  | 2.104  | –     | 14.7×     | –          |
| nyr8k | 4 | 0.05       | 68/416           | 5,713  | 195,594 | 614,621   | 37.970  | 1.758  | –     | 21.6×     | –          |
| nyr8k | 4 | 0.01       | 482/3,809        | 5,713  | 116,538 | 384,515   | 81.573  | 3.922  | 1.934 | 20.8×     | 42.2×      |

**Table 12.** Small NY instances and replications (BASE one run, FAST minimum of two runs; nyr2k at  $M = 3$  comes from a separate session). All rows are identical.

| graph | M | $\epsilon$            | $ T $  | exp                           | BASE                              | FAST                              | BASE/FAST     |
|-------|---|-----------------------|--------|-------------------------------|-----------------------------------|-----------------------------------|---------------|
| nyr2k | 3 | 0 / 0.01 / 0.05 / 0.1 | 109    | 1,903 / 1,960 / 2,294 / 2,525 | 0.0078 / 0.0060 / 0.0053 / 0.0057 | 0.0067 / 0.0051 / 0.0051 / 0.0050 | 1.04–1.18×    |
| nyr1k | 5 | 0.1 / 0.05 / 0.01     | 266    | 2,627 / 2,510 / 2,398         | 0.0082 / 0.0083 / 0.0081          | 0.0061 / 0.0070 / 0.0059          | 1.19–1.37×    |
| nyr2k | 5 | 0.1 / 0.05 / 0.01     | 680    | 8,769 / 7,518 / 6,831         | 0.0629 / 0.0553 / 0.0651          | 0.0259 / 0.0259 / 0.0308          | 2.11–2.43×    |
| nyr1k | 6 | 0.1 / 0.05 / 0.01     | 234    | 2,320 / 2,114 / 2,021         | 0.0120 / 0.0139 / 0.0102          | 0.0063 / 0.0085 / 0.0067          | 1.52–1.90×    |
| nyr2k | 6 | 0.1 / 0.05            | 2,051  | 24,873 / 22,778               | 0.8868 / 0.9524                   | 0.1729 / 0.1638                   | 5.13–5.81×    |
| nyr3k | 5 | 0.1 / 0.05            | 4,313  | 55,894 / 48,728               | 14.910 / 23.264                   | 0.726 / 0.462                     | 20.5× / 50.4× |
| nyr5k | 5 | 0.1                   | 26,701 | 618,328                       | 214.322                           | 11.578                            | 18.5×         |
| nyr3k | 6 | 0.1                   | 11,286 | 140,935                       | 120.112                           | 1.616                             | 74.3×         |

**Table 13.** Instances on which BASE exceeded the 900 s limit (FAST only).

| graph | M | $\epsilon$ | $ H(s) $ avg/max | $ T $   | exp       | gen        | FAST (s) | BASE/FAST |
|-------|---|------------|------------------|---------|-----------|------------|----------|-----------|
| nyr8k | 5 | 0.05       | 290/3,285        | 57,902  | 1,654,142 | 4,798,173  | 74.1     | > 12.2×   |
| nyr8k | 5 | 0.01       | 2,695/32,966     | 57,902  | 1,093,051 | 3,282,468  | 96.6     | > 9.3×    |
| nyr5k | 6 | 0.05       | 524/5,173        | 184,366 | 2,317,321 | 5,996,327  | 93.3     | > 9.6×    |
| nyr8k | 6 | 0.1        | 310/3,325        | 334,791 | 7,282,434 | 18,251,843 | 371.8    | > 2.4×    |
| nyr8k | 6 | 0.05       | 1,421/15,646     | 334,791 | 6,451,655 | 16,879,237 | 497.0    | > 1.8×    |

FAST is 2.6–4.7× faster than BASE on nyr3k at  $M = 4$ , where  $|T| = 646$ ; 4.1–5.6× on nyr5k and 14.7–21.6× on nyr8k at  $M = 4$ ; 21.0–60.9× at  $M = 5$ ; and 66.7–89.2× at  $M = 6$ . FAST2, measured in the same session as BASE, is 6.8–84.2× faster. On five instances BASE exceeded 900 s, and FAST returned fronts of up to 334,791 vectors after up to 7.3 million expansions (Table 13). Small instances ( $|T| \leq 680$ ) gain 1.0–2.4× (Table 12): the advantage requires large frontiers. Replicated speedups differ from Table 11 by –17% to +26%, mostly through the variance of the single BASE runs.

## 5.5 Goal-witness cache

**Table 14.** FAST2 versus FAST. Upper part: the same prototype executable with and without the cache (minimum of three runs, two at  $M = 6$ ). Lower part: FAST2 prototype versus the ported FAST (different executables, single runs, concurrent load).

| graph | M | $\epsilon$ | FAST (s) | FAST2 (s) | change | selection share of time (%) | goal queries answered by the cache |
|-------|---|------------|----------|-----------|--------|-----------------------------|------------------------------------|
| nyr3k | 5 | 0.01       | 0.680    | 0.423     | -38%   | 60.7 → 35.6                 | 91%                                |
| nyr3k | 5 | 0.05       | 0.524    | 0.424     | -19%   | 37.7 → 22.8                 | 82%                                |
| nyr8k | 4 | 0.05       | 2.736    | 2.330     | -15%   | 20.5 → 15.0                 | 77%                                |
| nyr3k | 6 | 0.1        | 1.963    | 1.726     | -12%   | 26.0 → 19.2                 | -                                  |
| nyr3k | 6 | 0.05       | 2.035    | 1.546     | -24%   | 32.5 → 23.5                 | -                                  |
| nyr5k | 4 | 0          | 5.307    | 3.236     | -39%   | -                           | -                                  |
| nyr8k | 4 | 0.01       | 3.922    | 1.934     | -51%   | -                           | -                                  |
| nyr3k | 6 | 0.01       | 3.381    | 2.680     | -21%   | -                           | -                                  |
| nyr5k | 5 | 0.01       | 10.127   | 9.099     | -10%   | -                           | -                                  |

In the same executable the cache reduces the running time by 12–38%. It answers 77–91% of the goal queries issued by heuristic selection, and the share of selection in the running time falls from 20.5–60.7% to 15.0–35.6%. The gain is largest where  $|H(s)|$  is large ( $\epsilon = 0.01$ ) or  $M$  is high. Adding a suffix ideal-point test on top of the cache brought no further gain. All rows return the same fronts, expansions, and generations as FAST.

## 5.6 Ablation

**Table 15.** Leave-one-out ablation: wall time of the configuration divided by that of FAST (prototype, minimum of three runs; > 1 means the mechanism pays).

| configuration                                          | 15×15, M=3 | 10×10, M=4 | 12×12, M=4 | 10×10, M=5 | 10×10, M=6 | A*pex 10×10, M=4 |
|--------------------------------------------------------|------------|------------|------------|------------|------------|------------------|
| layout only (no FAST mechanism)                        | 1.72       | 3.03       | 3.95       | 7.68       | 14.60      | 1.99             |
| – reordering                                           | 1.21       | 1.20       | 1.23       | 1.19       | 1.49       | 1.87             |
| – witness capture                                      | 1.03       | 1.29       | 1.25       | 1.36       | 1.10       | 1.11             |
| – redundancy pointers                                  | 1.06       | 0.99       | 1.01       | 1.09       | 0.94       | 0.96             |
| – frontier trees                                       | 1.02       | 1.59       | 1.84       | 2.90       | 6.31       | 1.29             |
| – contiguous $\text{Tr}(H(s))$ and redundancy pointers | 1.03       | 1.26       | 1.02       | 1.19       | 1.01       | 0.98             |
| fixed promotion size 16 instead of the cost model      | 1.12       | 1.07       | 0.94       | 1.12       | 0.96       | 0.99             |
| + static heuristic index ( $\theta_H = 1024$ )         | 1.03       | 1.16       | 0.97       | 0.94       | 0.96       | 0.92             |
| + Pareto pruning of $G_{cl}(s)$                        | 1.29       | 1.98       | 1.78       | 3.28       | 1.85       | 1.13             |
| + retesting the last dominator first                   | 1.03       | 1.03       | 1.31       | 1.03       | 0.98       | 0.92             |

Frontier trees dominate at high  $M$ : removing them costs 1.59–1.84× at  $M = 4$ , 2.90× at  $M = 5$ , and 6.31× at  $M = 6$ , and nothing at  $M = 3$ . Removing reordering costs 1.19–1.49× on landmark grids and 1.87× on the A\*pex grid; removing witness capture costs 1.03–1.36×. Redundancy pointers alone are within noise once the other mechanisms are active. The cost model matches a fixed size threshold within noise without per-dimension tuning. Three candidate mechanisms were rejected: the static heuristic index (0.92–1.16), Pareto pruning of  $G_{cl}(s)$ , which is harmful (1.13–3.28), and retesting the last dominator first (0.92–1.31). Together the FAST mechanisms make the search 1.7–14.6× faster than the layout-only control.

## 5.7 Verification of bit-identical Pareto sets

We re-verified all 274 archived solution files independently of the archived protocol. For every file we computed the SHA-1 digest of its sorted set of unique cost vectors, counted duplicate lines, and tested exactly whether the set is an antichain under weak dominance; the three largest sets, with 101,242 to 334,791 vectors, were tested by a blockwise comparison of each vector with its lexicographic predecessors, since a weak dominator is lexicographically smaller.

**Table 16.** Re-verification of archived Pareto sets.

| check                       | scope                                                                                                                                                                                                          | result                                                                 |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| agreement across solvers    | 81 valid inputs (instance $\times$ heuristic file $\times$ scan order; reordered A*pex files excluded) solved by two or more solvers, 230 files; $M = 3$ : 21, $M = 4$ : 35, $M = 5$ : 15, $M = 6$ : 10 inputs | identical unique sets on all 81                                        |
| agreement across heuristics | 14 NY instances; all $\varepsilon$ , solvers, and sessions                                                                                                                                                     | one front per instance, including the three instances only FAST solved |
| antichain property (exact)  | every output on a valid input, up to 334,791 vectors                                                                                                                                                           | all are antichains                                                     |
| duplicate cost vectors      | 274 files                                                                                                                                                                                                      | none                                                                   |
| invalid inputs              | reordered A*pex files (Table 3); legacy inadmissible landmark files                                                                                                                                            | outputs are not antichains                                             |

The three instances that only FAST solved are cross-checked through heuristics: on nyr8k at  $M = 5$  the heuristics with  $\varepsilon = 0.05$  and  $\varepsilon = 0.01$  lead to the same 57,902 vectors, and on nyr8k at  $M = 6$  those with  $\varepsilon = 0.1$  and  $\varepsilon = 0.05$  lead to the same 334,791 vectors; nyr5k at  $M = 6$  was solved with one heuristic only. On the reordered A\*pex input of Table 3, BASE and FAST agree with each other on the same incorrect set of 16,993 vectors, which is consistent with Theorem 2: FAST2 reproduces the baseline even on inputs that violate the precondition of Theorem 1.

## 5.8 Legacy and excluded data

Early sweeps used a landmark generator that ran Dijkstra on the directed edges, whereas the solver searches the symmetrised multigraph; the resulting vectors are not lower bounds. On the  $M = 4$  grid whose true front has 12,667 vectors, this generator produced fronts of 8,592–11,510 vectors, depending on  $K$ , and all seven archived baseline outputs on such files that we examined contain dominated vectors. Results on these files test solver equivalence on identical inputs, not MVH search. Table 17 reports the most informative of them, for completeness.

**Table 17.** Legacy grids with inadmissible landmark vectors: BASE versus DUAL (single runs; comparisons as counted by the instrumented solvers).

| grid           | M | $\rho$ | K       | output size                   | BASE (s)   | DUAL (s)   | BASE/DUAL                   | comparisons BASE / DUAL |
|----------------|---|--------|---------|-------------------------------|------------|------------|-----------------------------|-------------------------|
| 15 $\times$ 15 | 3 | -0.6   | 200–500 | 2,842                         | 40.5–122.3 | 41.9–185.4 | 0.60–1.00 $\times$ (7 runs) | 237–260 M / 58 M        |
| 10 $\times$ 10 | 4 | 0.0    | 50      | 1,889                         | 18.48      | 5.68       | 3.25 $\times$               | 165 M / 3.3 M           |
| 10 $\times$ 10 | 4 | -0.2   | 100     | 5,080                         | 78.99      | 14.32      | 5.52 $\times$               | 978 M / 8.7 M           |
| 10 $\times$ 10 | 4 | -0.4   | 100     | 9,480                         | 167.26     | 45.81      | 3.65 $\times$               | 1.84 G / 16.1 M         |
| 10 $\times$ 10 | 5 | 0.0    | 100     | 23,150                        | 525.00     | 110.22     | 4.76 $\times$               | 7.81 G / 48.7 M         |
| 10 $\times$ 10 | 5 | -0.2   | 50      | 48,303                        | 1,611.32   | 434.97     | 3.70 $\times$               | 14.0 G / 92.3 M         |
| 8 $\times$ 8   | 6 | 0.0    | 50      | 71,406 (BASE) / 71,191 (DUAL) | 2,286.31   | 382.14     | 5.98 $\times$               | 36.9 G / 78.9 M         |

On these grids the output size varies with  $K$  on the same graph (for example 23,998–48,303 at  $M = 5$ ,  $\rho = -0.2$ ), a symptom of inadmissibility. The  $M = 6$  run returned 71,406 solutions for BASE and 71,191 unique vectors for DUAL; an earlier audit attributed the difference to repeated cost vectors in the baseline output, but no solution file of that run was archived, and we could not re-verify it. We also excluded (i) all runs on an  $8 \times 8$  grid whose five-column A\*pex file was paired with  $M = 5$  in some sessions and with  $M = 6$  in others, the latter reading past the end of every heuristic vector, and (ii) two earlier in-house variants of the static index, one of which read past the end of  $H(s)$  and one of which returned a synthetic vector not in  $H(s)$  and was not identical to BASE.

## 5.9 Speedup scaling

**Table 18.** Speedup of FAST over BASE on the NY instances of Table 11, by dimension.

| M | rows | min   | geometric mean | max   |
|---|------|-------|----------------|-------|
| 4 | 11   | 2.6×  | 6.0×           | 21.6× |
| 5 | 6    | 21.0× | 31.0×          | 60.9× |
| 6 | 3    | 66.7× | 76.5×          | 89.2× |

Across the 20 rows of Table 11, the speedup correlates more with the baseline’s time per expansion (Spearman  $\rho = 0.93$ ) and with  $M$  (0.88) than with  $|T|$  (0.50) or with the mean size of  $H(s)$  (0.41). The baseline’s cost per expansion is proportional to the sizes of the frontiers it scans linearly; FAST’s grows sublinearly in them. This refines the whole-search argument of Shperberg et al. (2026), by which speedup follows the distribution of frontier sizes rather than the graph size. Two examples: nyr8k at  $M = 4$  ( $|T| = 5,713$ ) gains 14.7–21.6× while nyr5k at  $M = 4$  ( $|T| = 7,105$ ) gains 4.1–5.6×, because at  $\varepsilon \in \{0.1, 0.05\}$  BASE spends 123–194  $\mu\text{s}$  per expansion on the former and 31–38  $\mu\text{s}$  on the latter. On the larger instances the speedup grows as  $\varepsilon$  decreases in five of the six series measured within one session (nyr3k,  $M = 4$ : 2.6×  $\rightarrow$  4.7×; nyr3k,  $M = 5$ : 24.6×  $\rightarrow$  60.9×; nyr3k,  $M = 6$ : 75.1×  $\rightarrow$  89.2×; nyr5k,  $M = 4$ : 4.1×  $\rightarrow$  5.6×; nyr8k,  $M = 4$ : 14.7×  $\rightarrow$  21.6×); the exception, nyr5k at  $M = 5$  (21.8×  $\rightarrow$  21.0×), is within noise. A smaller  $\varepsilon$  gives larger sets  $H(s)$  and fewer expansions; BASE pays for every refuted candidate, FAST for few. Small instances (Table 12) show no consistent trend.

## 5.10 Threats to validity

- **Timing.** All measurements come from one laptop with thermal throttling; the baseline was run once per configuration; the long New York sweep and the cache runs shared the machine with other runs. On landmark grids FAST and FAST with  $\theta_H = 1024$  execute the same code, because no state has 1,024 heuristic vectors, yet their times differ by up to 1.5×; this bounds the noise of Table 9.
- **Two implementations.** FAST was measured both as a ported solver and as a prototype; FAST2 exists only as a prototype. The lower part of Table 14 compares different executables.
- **Coverage.** Grids have 100–225 vertices, and the road instances are subgraphs of one road network with up to 8,000 vertices. Five instances have no baseline time; their correctness rests on the cross-heuristic agreement and the antichain tests of Section 5.7.
- **Identity.** Trajectory identity is defined per standard library (Section 3.4); all executables used libc++.

## 6 Discussion and Future Work

### 6.1 Findings

1. **Geometric indexing pays when the node test is cheap relative to the work it prunes.** Dynamic frontier trees answer point queries with  $d$  comparisons per node. They win from  $M = 4$  on (1.6–6.3× in the ablation), as the separation of Shperberg et al. (2026) predicts, and at  $M = 3$  a contiguous scan is as good, which the cost model detects. Static trees over  $H(s)$  answer set queries with one frontier query per node. They pay only when refutation runs are long, which reordering largely prevents, and they cannot shorten certification, because their accept rule almost never fires.
2. **Operation order matters as much as data structures.** Testing local dominance before heuristic selection is free, because both tests are pure, and it removes 67–80% of the selection calls. It reduces the gain of the static heuristic index from 1.3–1.9× to about 1.1× at best.

3. **The semantics of the heuristic decides the algorithm.** Landmark families are universally valid; they are better aggregated by component-wise maximum (11–41% fewer expansions), and their scan order is irrelevant. Genuine MVHs must be evaluated existentially, and under DR each  $H(s)$  must be scanned in non-decreasing order of  $h_1$ ; other selection rules can lose Pareto-optimal solutions or return dominated ones. Alternative orderings of  $H(s)$  therefore require a different ordering function or the removal of DR (Table 3).
4. **The remaining cost lies mostly outside dominance tests.** With FAST’s mechanisms in place, OPEN operations, node allocation, and successor generation take 45–70% of the running time on landmark grids with  $M \leq 5$ , 26% at  $M = 6$ , and 15% on the A\*pex grid. Among dominance operations, the local test is the largest cost on landmark grids (20–43%), and heuristic selection on the A\*pex grid (65% before the goal-witness cache).

## 6.2 Future work

1. **Min-ordered MVH search.** C-MVH-Min\* (Anonymous 2026b) replaces the lexicographic ordering function with Min (Skyler et al. 2022, 2026a). Under Min, the key of a node’s whole evaluation set is obtained exactly from the ideal point of  $H(s)$ , ordering is decoupled from filtering, and the lexicographic precondition of Theorem 1 disappears. Filtering becomes an existential survival query over a full-dimensional index of  $H(s)$ : a set-against-set query of the kind analysed in Section 4, but one that needs *any* surviving member rather than the first in a fixed order, so a traversal may visit the most promising subtree first. Preliminary bi-objective evidence shows the pruning benefit of MVHs saturating at 8–16 vectors per state (93–98% of the perfect-MVH ceiling), which favours compact indexed heuristic sets. The natural combination is a frontier index over  $F(s)$  with a heuristic index over  $H(s)$  under Min ordering; constant-time dominance checks for Min beyond two objectives (Skyler et al. 2026a, 2026b) are the open prerequisite.
2. **DR-free lazy search.** Without DR, the selection order no longer affects correctness (Table 3), which admits ideal-point-guided selection. The file order remained the cheapest order tested, however, and the archived DR-free prototype, which scans all sets linearly, was 40× slower than FAST2 at equal expansions. A DR-free search needs full-dimensional frontier indices to be competitive.
3. **Engineering.** An arena of fixed-size nodes with an index-based heap that preserves the tie-breaking sequence; porting the goal-witness cache into the ported solver; dimension-specific leaf capacities and branch-free kernels for the local test at  $M = 6$ , where it takes 43% of the time; and SIMD dominance kernels, which made the byte-identical A\*pex builder 6–26× faster.
4. **Broader evaluation.** Several road networks,  $M \geq 7$ , several machines, and direct cache-miss measurements to test the locality attribution of Section 4.5.

---

## References

- Anonymous. 2026a. Efficient Multi-Valued Heuristic Search via KD-Tree Dominance Pruning (KD-ChooseH). Anonymized manuscript under review.
- Anonymous. 2026b. Compact Multi-Valued Heuristic Search under Min Ordering (C-MVH-Min\*). Anonymized manuscript under review.
- Demetrescu, C.; Goldberg, A. V.; and Johnson, D. S., eds. 2009. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*. DIMACS Series in Discrete Mathematics and Theoretical Computer Science 74. American Mathematical Society.
- Felner, A.; Goldenberg, M.; Sharon, G.; Stern, R.; Beja, T.; Sturtevant, N.; Schaeffer, J.; and Holte, R. C. 2012. Partial-Expansion A\* with Selective Node Generation. In *Proceedings of AAAI*.

- Geißer, F.; Haslum, P.; Thiébaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. In *Proceedings of ICAPS*, 100–109.
- Goldberg, A. V.; and Harrelson, C. 2005. Computing the Shortest Path: A\* Search Meets Graph Theory. In *Proceedings of SODA*, 156–165.
- Hernández, C.; Yeoh, W.; Baier, J. A.; Zhang, H.; Suazo, L.; Koenig, S.; and Salzman, O. 2023a. Simple and Efficient Bi-Objective Search Algorithms via Fast Dominance Checks. *Artificial Intelligence* 314: 103807.
- Hernández, C.; Yeoh, W.; Baier, J. A.; Felner, A.; Salzman, O.; Zhang, H.; Chan, S.-H.; and Koenig, S. 2023b. Multi-Objective Search via Lazy and Efficient Dominance Checks. In *Proceedings of IJCAI*.
- Lee, D. T.; and Wong, C. K. 1977. Worst-Case Analysis for Region and Partial Region Searches in Multidimensional Binary Search Trees and Balanced Quad Trees. *Acta Informatica* 9(1): 23–29.
- Madow, L.; and Pérez de la Cruz, J. L. 2005. A New Approach to Multiobjective A\* Search. In *Proceedings of IJCAI*, 218–223.
- Madow, L.; and Pérez de la Cruz, J. L. 2010. Multiobjective A\* Search with Consistent Heuristics. *Journal of the ACM* 57(5): 27:1–27:25.
- Pulido, F. J.; Madow, L.; and Pérez-de-la-Cruz, J. L. 2015. Dimensionality Reduction in Multiobjective Shortest Path Search. *Computers & Operations Research* 64: 60–70.
- Ren, Z.; Hernández, C.; Likhachev, M.; Felner, A.; Koenig, S.; Salzman, O.; Rathinam, S.; and Choset, H. 2025. EMOA\*: A Framework for Search-Based Multi-Objective Path Planning. *Artificial Intelligence* 339: 104260.
- Salzman, O.; Felner, A.; Hernández, C.; Zhang, H.; Chan, S.-H.; and Koenig, S. 2023. Heuristic-Search Approaches for the Multi-Objective Shortest-Path Problem: Progress and Research Opportunities. In *Proceedings of IJCAI*, 6759–6768.
- Salzman, O.; Hernández Ulloa, C.; Felner, A.; and Koenig, S. 2026. Multi-Objective Search: Algorithms, Applications, and Emerging Directions. In *Proceedings of AAAI*, 40990–40999.
- Shperberg, S. S.; et al. 2026. A Geometric Index for Multi-Objective Dominance Checking. Anonymized manuscript under review (authorship as recorded in the project’s literature archive).
- Skyler, S.; Atzmon, D.; Felner, A.; Salzman, O.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernández, C. 2022. Bounded-Cost Bi-Objective Heuristic Search. In *Proceedings of SoCS*, 239–243.
- Skyler, S.; Shperberg, S. S.; Atzmon, D.; Felner, A.; Salzman, O.; Chan, S.; Zhang, H.; Koenig, S.; Yeoh, W.; and Hernández Ulloa, C. 2024. Theoretical Study on Multi-Objective Heuristic Search. In *Proceedings of IJCAI*, 7021–7028.
- Skyler, S.; Atzmon, D.; Felner, A.; Shperberg, S. S.; Salzman, O.; Hernández Ulloa, C.; and Koenig, S. 2026a. Theoretical Study on Multi-Objective Heuristic Search. *Journal of Artificial Intelligence Research* 4, Article 6.
- Skyler, S.; Atzmon, D.; Felner, A.; Salzman, O.; Hernández Ulloa, C.; and Koenig, S. 2026b. Deeper Treatment of the Bi-Objective Search Framework. In *Proceedings of AAAI*.
- Stern, R.; Goldenberg, M.; Saffidine, A.; and Felner, A. 2021. Heuristic Search for One-to-Many Shortest Path Queries. *Annals of Mathematics and Artificial Intelligence*. doi:10.1007/s10472-021-09775-x.
- Stewart, B. S.; and White, C. C., III. 1991. Multiobjective A\*. *Journal of the ACM* 38(4): 775–814.
- Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. In *Proceedings of SoCS*.

Zhang, H.; Salzman, O.; Kumar, T. K. S.; Hernández Ulloa, C.; Suazo, L.; and Koenig, S. 2022. A\*pex: Efficient Approximate Multi-Objective Search on Graphs. In *Proceedings of ICAPS*, 394–403.

Zhang, H.; Salzman, O.; Felner, A.; Kumar, T. K. S.; Skyler, S.; Hernández Ulloa, C.; and Koenig, S. 2023. Towards Effective Multi-Valued Heuristics for Bi-Objective Shortest-Path Algorithms via Differential Heuristics. In *Proceedings of SoCS*, 101–109.

## Appendix A: Artifacts

All paths are relative to the worktree of the named branch. Directories under `benchmarks/runs/` are ignored by version control and exist only in the local worktrees; everything else listed on the branch `barfe-aicahub-mvh-kdtree-perf-audit` is committed (commit 54711d8).

| artifact                                                  | branch                        | path                                                                                                                                                                                                                |
|-----------------------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reference L-NAMOA*dr-mvh                                  | all                           | baselines/bridging-mvh-dr (commit 0a2f9ea, read-only)                                                                                                                                                               |
| ported FAST solver                                        | mvh-kdtree-perf-audit         | scratchpad/proposal/src/(l_namoa_dr_mvh_fast.cpp, hybrid_frontier.h, heuristic_index.h); harness scratchpad/perf/harness.cpp                                                                                        |
| FAST prototype                                            | mvh-kdtree-perf-audit         | scratchpad/perf/proto.hpp                                                                                                                                                                                           |
| FAST2 prototype                                           | mvh-kdtree-perf-audit         | scratchpad/perf/proto_w.hpp (option tcache=2)                                                                                                                                                                       |
| DR-free order probe                                       | mvh-kdtree-perf-audit         | scratchpad/perf/nodr.cpp                                                                                                                                                                                            |
| A*pex builder re-implementation                           | mvh-kdtree-perf-audit         | scratchpad/apexopt/fast_apex_mvh2.hpp                                                                                                                                                                               |
| prototype sweeps E1–E4, per-size profile, BASE references | mvh-kdtree-perf-audit         | scratchpad/runs/{e1,e2,e3,e3b,e3c,e4}.csv, prof_tables.txt, ref/, refA/                                                                                                                                             |
| grid head-to-head (Table 9)                               | mvh-kdtree-perf-audit         | benchmarks/runs/2026-09-25_124816/                                                                                                                                                                                  |
| landmark-set size, order experiment (Tables 1, 3)         | mvh-kdtree-perf-audit         | benchmarks/runs/2026-09-28_093218/, benchmarks/runs/2026-09-28_095828/                                                                                                                                              |
| static-index sweep in FAST order (Table 6)                | mvh-kdtree-perf-audit         | benchmarks/runs/2026-09-28_112941_p2_roi/                                                                                                                                                                           |
| NY runs (Tables 11, 12)                                   | mvh-kdtree-perf-audit         | benchmarks/runs/{2026-09-28_113753_p3_ny, 2026-09-28_121517_ny_hiM, marathon_ny_hiM}/                                                                                                                               |
| goal-witness cache, DR-free test (Tables 3, 14)           | mvh-kdtree-perf-audit         | benchmarks/runs/{2026-09-28_1650_tcache, 2026-09-28_1830_nodr}/; summaries in docs/theory/agent2_*                                                                                                                  |
| formerly unbuildable NY instances (Tables 11, 13, 14)     | optimize-apex-heuristic-build | benchmarks/runs/2026-09-28_175731_timeouts_fastbuild/                                                                                                                                                               |
| legacy sweeps (Table 17)                                  | main                          | benchmarks/runs/adaptive_explorer/                                                                                                                                                                                  |
| re-verification and consolidated tables (Section 5.7)     | fast2-research-paper          | benchmarks/runs/2026-09-29_095221_paper_verification/(fronts.csv, equality.md, tables.md, exact_antichain.csv, heuristic_structure.csv, args.txt); scripts in scratchpad/paper_verify/ (ignored by version control) |

## Appendix B: Default configuration

| parameter                                            | value                                                                                                |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| promotion size $\theta_n$ (local and goal frontiers) | 8                                                                                                    |
| promotion scan cost $\theta_c$                       | 64 comparisons per query, running mean over at least 16 queries, halved every 4,096 queries          |
| k-d rebuild                                          | when the node count exceeds $\alpha = 1.25$ times the live count at the last build plus $\beta = 64$ |
| static heuristic index                               | $\theta_H = \infty$ (disabled); leaf size 8 when enabled                                             |
| redundancy pointers                                  | built on first use; construction skipped when the first 48 vectors of $H(s)$ contain no redundancy   |
| goal-witness cache                                   | one slot per $(s, i)$ plus the last witness of the current call (FAST2 only)                         |
| prototype flags (FAST / FAST2)                       | flatH=1 local_first=1 witness=1 redund=1 localX=8 targetX=8 promoteC=64 / same with tcache=2         |
| compiler                                             | clang 21.1, libc++, -std=c++20 -O2 -DNDEBUG -march=native                                            |